

CHAPTER 10

PRINCIPLES OF DIMENSIONAL MODELING

CHAPTER OBJECTIVES

- Clearly understand how the requirements definition determines data design
- Introduce dimensional modeling and contrast it with entity-relationship modeling
- Review the basics of the STAR schema
- Find out what is inside the fact table and inside the dimension tables
- Determine the advantages of the STAR schema for data warehouses
- Review examples of the STAR schema

FROM REQUIREMENTS TO DATA DESIGN

The requirements definition completely drives the data design for the data warehouse. Data design consists of putting together the data structures. A group of data elements form a data structure. Logical data design includes determination of the various data elements that are needed and combination of the data elements into structures of data. Logical data design also includes establishing the relationships among the data structures.

Let us look at Figure 10-1. Notice how the phases start with requirements gathering. The results of the requirements gathering phase is documented in detail in the requirements definition document. An essential component of this document is the set of information package diagrams. Remember that these are information matrices showing the metrics, business dimensions, and the hierarchies within individual business dimensions.

The information package diagrams form the basis for the logical data design for a data warehouse made up of data marts. The data design process results in a dimensional data model.

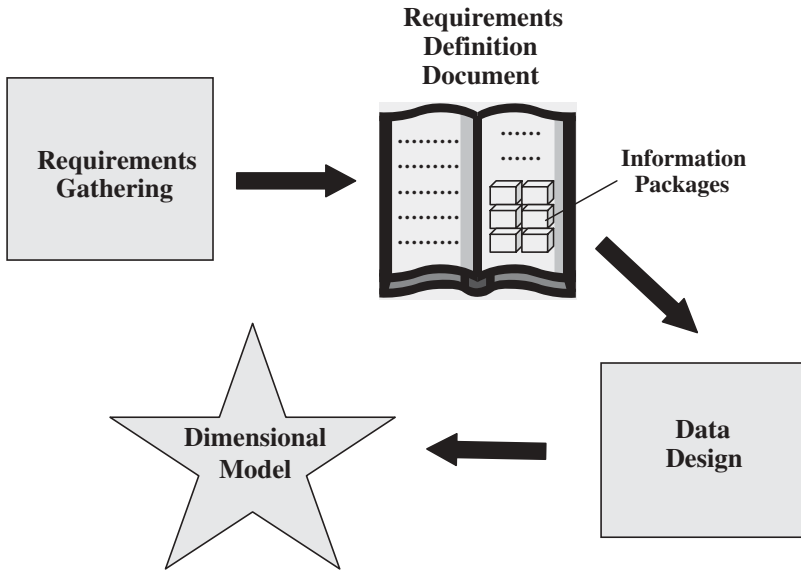


Figure 10-1 From requirements to data design.

Design Decisions

Before we proceed with designing the dimensional data model, let us quickly review some of the design decisions you have to make:

Choosing the Process. Selecting the subjects from the information packages for the first set of logical structures to be designed.

Choosing the Grain. Determining the level of detail for the data in the data structures.

Identifying and Conforming the Dimensions. Choosing the business dimensions (such as product, market, time, etc.) to be included in the first set of structures and making sure that each particular data element in every business dimension is conformed to one another.

Choosing the Facts. Selecting the metrics or units of measurements (such as product sale units, dollar sales, dollar revenue, etc.) to be included in the first set of structures.

Choosing the Duration of the Database. Determining how far back in time you should go for historical data.

Dimensional Modeling Basics

Dimensional modeling gets its name from the business dimensions we need to incorporate into the logical data model. It is a logical design technique to structure the business dimensions and the metrics that are analyzed along these dimensions. This modeling technique is intuitive for that purpose. The model has also proved to provide high performance for queries and analysis.

The multidimensional information package diagram we have discussed is the foundation for the dimensional model. Therefore, the dimensional model consists of the specific data

structures needed to represent the business dimensions. These data structures also contain the metrics or facts.

In Chapter 5, we discussed information package diagrams in sufficient detail. We specifically looked at an information package diagram for automaker sales. Please go back and review Figure 5-5 in that chapter. What do you see? In the bottom section of the diagram, you observe the list of measurements or metrics that the automaker wants to use for analysis. Next, look at the column headings. These are the business dimensions along which the automaker wants to analyze the measurements or metrics. Under each column heading you see the dimension hierarchies and categories within that business dimension. What you see under each column heading are the attributes relating to that business dimension.

Reviewing the information package diagram for automaker sales, we notice three types of data entities: (1) measurements or metrics, (2) business dimensions, and (3) attributes for each business dimension. So, when we put together the dimensional model to represent the information contained in the automaker sales information package, we need to come up with data structures to represent these three types of data entities. Let us discuss how we can do this.

First, let us work with the measurements or metrics seen at the bottom of the information package diagram. These are the facts for analysis. In the automaker sales diagram, the facts are as follows:

Actual sale price
MSRP
Options price
Full price
Dealer add-ons
Dealer credits
Dealer invoice
Amount of down payment
Manufacturer proceeds
Amount financed

Each of these data items is a measurement or fact. Actual sale price is a fact about what the actual price was for the sale. Full price is a fact about what the full price was relating to the sale. As we review each of these factual items, we find that we can group all of these into a single data structure. In relational database terminology, you may call the data structure a relational table. So, the metrics or facts from the information package diagram will form the fact table. For the automaker sales analysis this fact table would be the automaker sales fact table.

Look at Figure 10-2 showing how the fact table is formed. The fact table gets its name from the subject for analysis; in this case, it is automaker sales. Each fact item or measurement goes into the fact table as an attribute for automaker sales.

We have determined one of the data structures to be included in the dimensional model for automaker sales and derived the fact table from the information package diagram. Let us now move on to the other sections of the information package diagram, taking the business dimensions, one by one. Look at the product business dimension in Figure 5-5.

The product business dimension is used when we want to analyze the facts by products. Sometimes our analysis could be a breakdown by individual models. Another analysis could

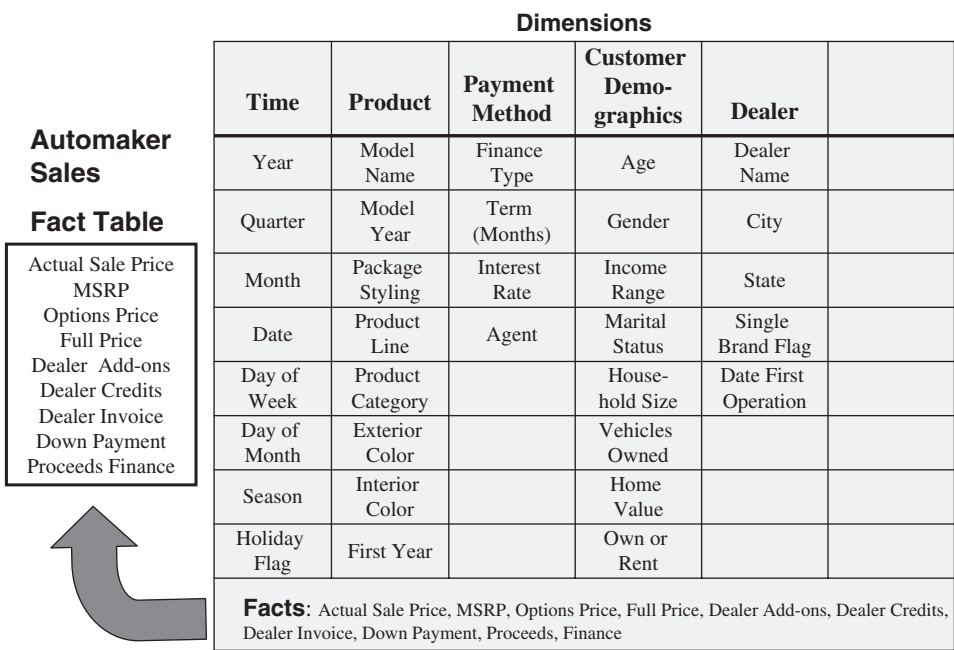


Figure 10-2 Formation of the automaker sales fact table.

be at a higher level by product lines. Yet another analysis could be at even a higher level by product categories. The list of data items relating to the product dimension are as follows:

- Model name
- Model year
- Package styling
- Product line
- Product category
- Exterior color
- Interior color
- First model year

What can we do with all these data items in our dimensional model? All of these relate to the product in some way. We can, therefore, group all of these data items in one data structure or one relational table. We can call this table the product dimension table. The data items in the above list would all be attributes in this table.

Looking further into the information package diagram, we note the other business dimensions shown as column headings. In the case of the automaker sales information package diagram, these other business dimensions are dealer, customer demographics, payment method, and time. Just as we formed the product dimension table, we can form the remaining dimension tables of dealer, customer demographics, payment method, and time. The data items shown within each column would then be the attributes for each corresponding dimension table.

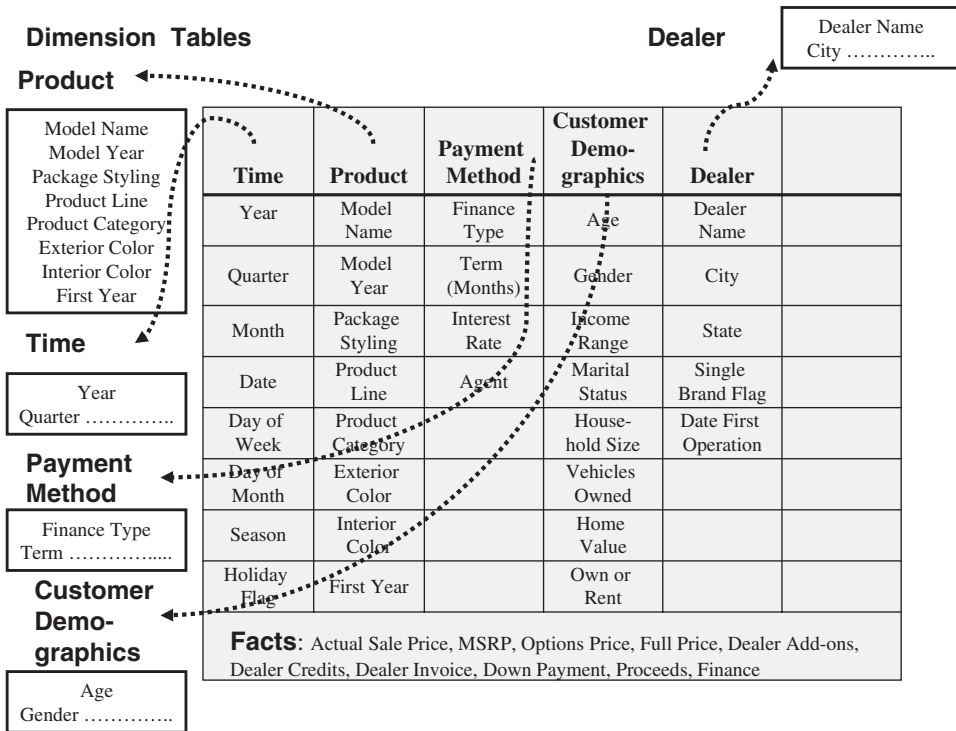


Figure 10-3 Formation of the automaker dimension tables.

Figure 10-3 puts all of this together. It shows how the various dimension tables are formed from the information package diagram. Look at the figure closely and see how each dimension table is formed.

So far we have formed the fact table and the dimension tables. How should these tables be arranged in the dimensional model? What are the relationships and how should we mark the relationships in the model? The dimensional model should primarily facilitate queries and analyses. What would be the types of queries and analyses? These would be queries and analyses where the metrics inside the fact table are analyzed across one or more dimensions using the dimension table attributes.

Let us examine a typical query against the automaker sales data. How much sales proceeds did the Jeep Cherokee, Year 2007 Model with standard options, generate in July 2007 at Big Sam Auto dealership for buyers who own their homes and who took 3-year leases, financed by Daimler-Chrysler Financing? We are analyzing actual sale price, MSRP, and full price. We are analyzing these facts along attributes in the various dimension tables. The attributes in the dimension tables act as constraints and filters in our queries. We also find that any or all of the attributes of each dimension table can participate in a query. Further, each dimension table has an equal chance to be part of a query.

Before we decide how to arrange the fact and dimension tables in our dimensional model and mark the relationships, let us go over what the dimensional model needs to achieve and what its purposes are. Here are some of the criteria for combining the tables into a dimensional model.

- The model should provide the best data access.
- The whole model must be query-centric.
- It must be optimized for queries and analyses.
- The model must show that the dimension tables interact with the fact table.
- It should also be structured in such a way that every dimension can interact equally with the fact table.
- The model should allow drilling down or rolling up along dimension hierarchies.

With these requirements, we find that a dimensional model with the fact table in the middle and the dimension tables arranged around the fact table satisfies the conditions. In this arrangement, each of the dimension tables has a direct relationship with the fact table in the middle. This is necessary because every dimension table with its attributes must have an even chance of participating in a query to analyze the attributes in the fact table.

Such an arrangement in the dimensional model looks like a star formation, with the fact table at the core of the star and the dimension tables along the spikes of the star. The dimensional model is therefore called a STAR schema.

Let us examine the STAR schema for the automaker sales as shown in Figure 10-4. The sales fact table is in the center. Around this fact table are the dimension tables of product, dealer, customer demographics, payment method, and time. Each dimension table is related to the fact table in a one-to-many relationship. In other words, for one row in the product dimension table, there are one or more related rows in the fact table.

E-R Modeling Versus Dimensional Modeling

We are familiar with data modeling for operational or OLTP systems. We adopt the Entity-Relationship (E-R) modeling technique to create the data models for these systems.

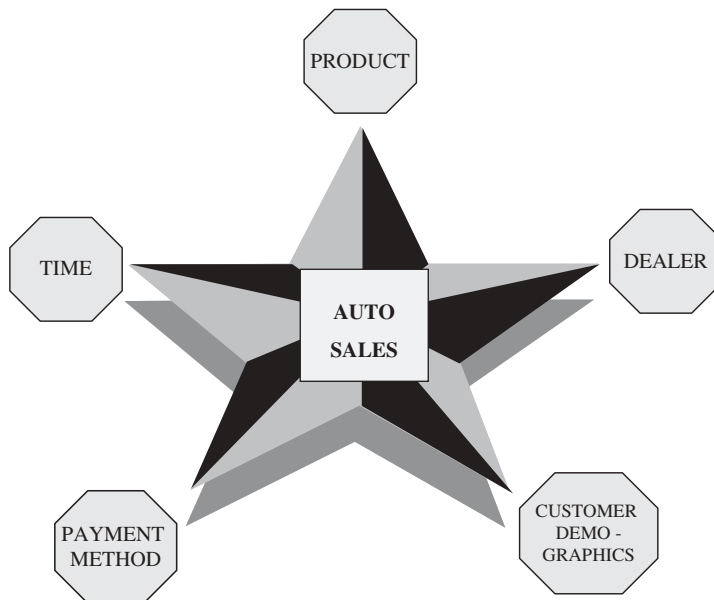


Figure 10-4 STAR schema for automaker sales.

- ◆ OLTP systems capture details of events or transactions
- ◆ OLTP systems focus on individual events
- ◆ An OLTP system is a window into micro-level transactions
- ◆ Picture at detail level necessary to run the business
- ◆ Suitable only for questions at transaction level
- ◆ Data consistency, non-redundancy, and efficient data storage critical

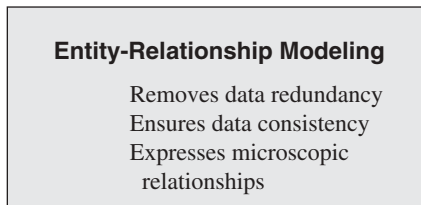


Figure 10-5 E-R modeling for OLTP systems

Figure 10-5 lists the characteristics of OLTP systems and shows why E-R modeling is suitable for OLTP systems.

We have so far discussed the basics of the dimensional model and find that this model is most suitable for modeling the data for the data warehouse. Let us recapitulate the characteristics of the data warehouse information and review how dimensional modeling is suitable for this purpose. Study the features of dimensional modeling listed in Figure 10-6.

- ◆ DW meant to answer questions on overall process
- ◆ DW focus is on how managers view the business
- ◆ DW reveals business trends
- ◆ Information is centered around a business process
- ◆ Answers show how the business measures the process
- ◆ The measures to be studied in many ways along several business dimensions

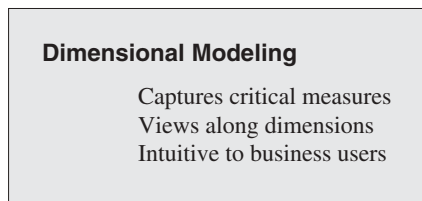


Figure 10-6 Dimensional modeling for the data warehouse.

Use of CASE Tools

Many case tools are available for data modeling. In Chapter 8, we introduced these tools and their features. You can use these tools for creating the logical schema and the physical schema for specific target database management systems (DBMSs).

You can use a case tool to define the tables, the attributes, and the relationships. You can assign the primary keys and indicate the foreign keys. You can form the entity-relationship diagrams. All of this is done very easily using graphical user interfaces and powerful drag-and-drop facilities. After creating an initial model, you may add fields, delete fields, change field characteristics, create new relationships, and make any number of revisions with utmost ease.

Another very useful function found in the case tools is the ability to forward-engineer the model and generate the schema for the target database system you need to work with. Forward-engineering is easily done with these case tools.

For modeling the data warehouse, we are interested in the dimensional modeling technique. Most of the existing vendors have expanded their modeling case tools to include dimensional modeling. You can create fact tables, dimension tables, and establish the relationships between each dimension table and the fact table. The result is a STAR schema for your model. Again, you can forward-engineer the dimensional STAR model into a relational schema for your chosen database management system.

THE STAR SCHEMA

Now that you have been introduced to the STAR schema, let us take a simple example and examine its characteristics in detail. Creating the STAR schema is the fundamental data design technique for the data warehouse. It is necessary to gain a good grasp of this technique.

Review of a Simple STAR Schema

We will take a simple STAR schema designed for order analysis. Assume this to be the schema for a manufacturing company and that the marketing department is interested in determining how they are doing with the orders received by the company.

Figure 10-7 shows this simple STAR schema. It consists of the orders fact table shown in the middle of the schema diagram. Surrounding the fact table are the four dimension tables of customer, salesperson, order date, and product. Let us begin to examine this STAR schema. Look at the structure from the point of view of the marketing department. The users in this department will analyze the orders using dollar amounts, cost, profit margin, and sold quantity. This information is found in the fact table of the structure. The users will analyze these measurements by breaking down the numbers in combinations by customer, salesperson, date, and product. All these dimensions along which the users will analyze are found in the structure. The STAR schema structure is a structure that can be easily understood by the users and with which they can comfortably work. The structure mirrors how the users normally view their critical measures along their business dimensions.

When you look at the order dollars, the STAR schema structure intuitively answers the questions of what, when, by whom, and to whom. From the STAR schema, the users can easily visualize the answers to these questions: For a given amount of dollars, what was

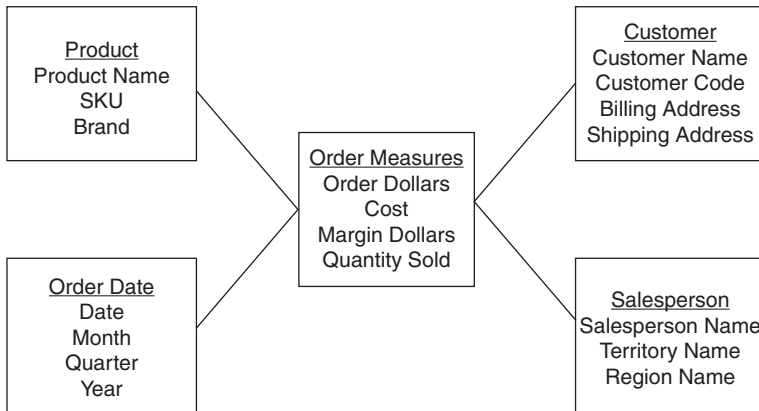


Figure 10-7 Simple STAR schema for orders analysis.

the product sold? Who was the customer? Which salesperson brought the order? When was the order placed?

When a query is made against the data warehouse, the results of the query are produced by combining or joining one or more dimension tables with the fact table. The joins are between the fact table and individual dimension tables. The relationship of a particular row in the fact table is with the rows in each dimension table. These individual relationships are clearly shown as the spikes of the STAR schema.

Take a simple query against the STAR schema. Let us say that the marketing department wants the quantity sold and order dollars for product *bigpart-1*, relating to customers in the

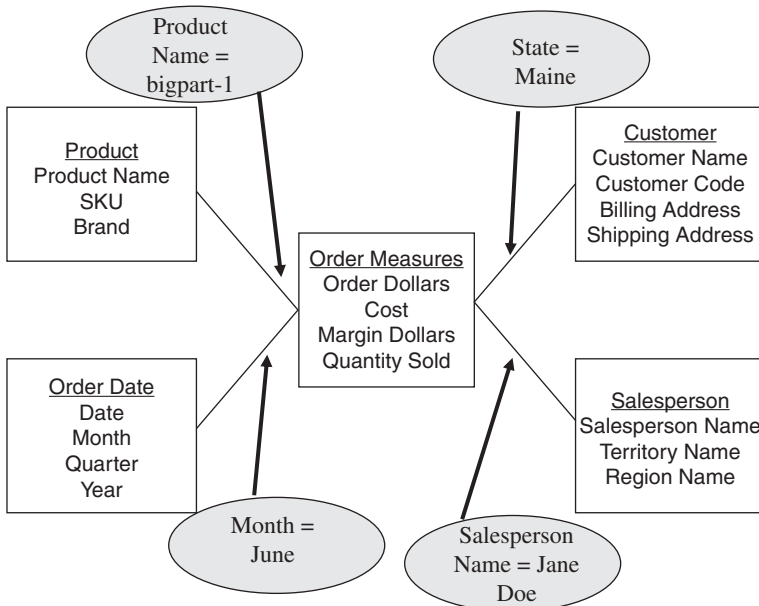


Figure 10-8 Understanding a query from the STAR schema.

state of *Maine*, obtained by salesperson *Jane Doe*, during the month of *June*. Figure 10-8 shows how this query is formulated from the STAR schema. Constraints and filters for queries are easily understood by looking at the STAR schema.

A common type of analysis is the drilling down of summary numbers to get at the details at the lower levels. Let us say that the marketing department has initiated a specific analysis by placing the following query: Show me the total quantity sold of product brand *big parts* to customers in the *northeast region for year 2008*. In the next step of the analysis, the marketing department now wants to drill down to the level of *quarters in 2008* for the *northeast region* for the same product brand, *big parts*. Next, the analysis goes down to the level of individual products in that brand. Finally, the analysis goes to the level of details by individual states in the northeast region. The users can easily discern all of this drill-down analysis by reviewing the STAR schema. Refer to Figure 10-9 to see how the drill-down is derived from the STAR schema.

Inside a Dimension Table

We have seen that a key component of the STAR schema is the set of dimension tables. These dimension tables represent the business dimensions along which the metrics are

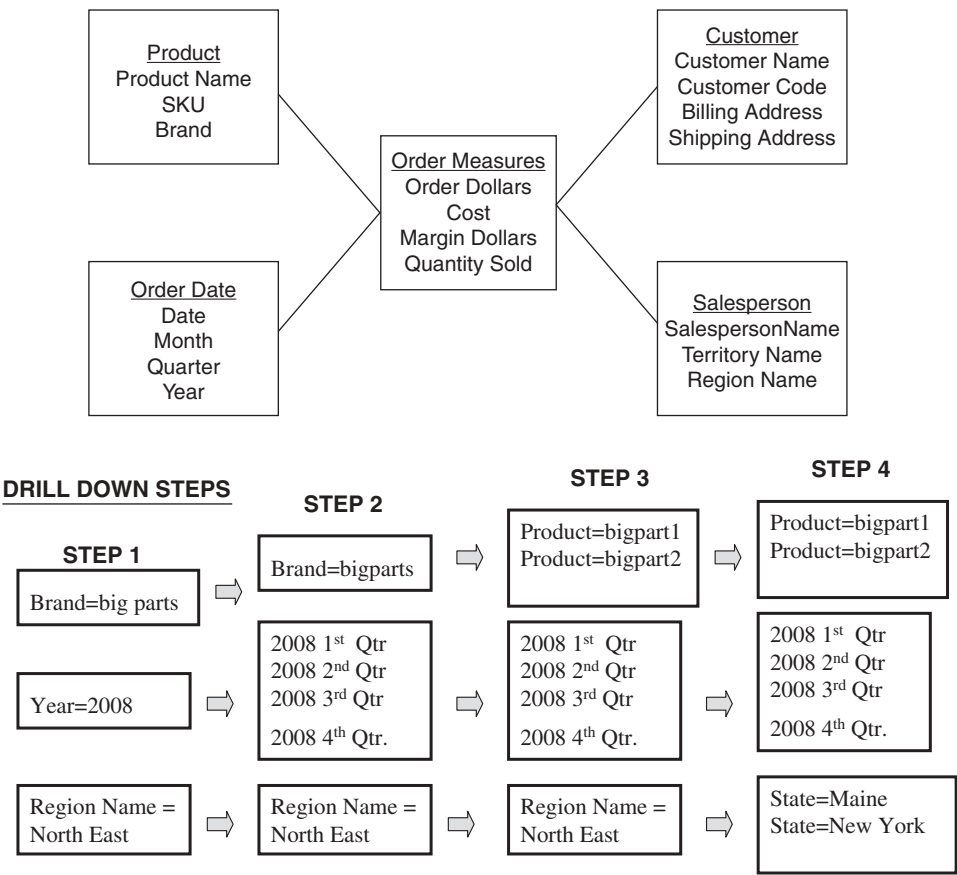


Figure 10-9 Understanding drill-down analysis from the STAR schema.

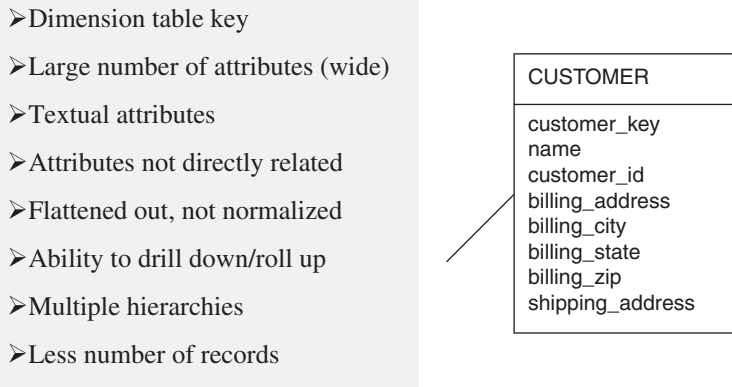


Figure 10-10 Inside a dimension table.

analyzed. Let us look inside a dimension table and study its characteristics. See Figure 10-10 and review the following observations.

Dimension Table Key. The primary key of the dimension table uniquely identifies each row in the table.

Table is Wide. Typically, a dimension table has many columns or attributes. It is not uncommon for some dimension tables to have more than 50 attributes. Therefore, we say that the dimension table is wide. If you lay it out as a table with columns and rows, the table is spread out horizontally.

Textual Attributes. In the dimension table you will seldom find any numerical values used for calculations. The attributes in a dimension table are of textual format.

These attributes represent the textual descriptions of the components within the business dimensions. Users will compose their queries using these descriptors.

Attributes not Directly Related. Frequently you will find that some of the attributes in a dimension table are not directly related to the other attributes in the table. For example, package size is not directly related to product brand; nevertheless, package size and product brand could both be attributes of the product dimension table.

Not Normalized. The attributes in a dimension table are used over and over again in queries. An attribute is taken as a constraint in a query and applied directly to the metrics in the fact table. For efficient query performance, it is best if the query picks up an attribute from the dimension table and goes directly to the fact table and not through other intermediary tables. If you normalize the dimension table, you will be creating such intermediary tables and that will not be efficient. Therefore, a dimension table is flattened out, not normalized.

Drilling Down, Rolling Up. The attributes in a dimension table provide the ability to get to the details from higher levels of aggregation to lower levels of details. For example, the three attributes zip code, city, and state form a hierarchy. You may get the total sales by state, then drill down to total sales by city, and then by zip code. Going the other way, you may first get the totals by zip codes, and then roll up to totals by city and state.

Multiple Hierarchies. In the example of the customer dimension, there is a single hierarchy going up from individual customer to zip, city, and state. But dimension tables often provide for multiple hierarchies, so that drilling down may be performed along

any of the multiple hierarchies. Take for example a product dimension table for a department store. In this business, the marketing department may have its way of classifying the products into product categories and product departments. On the other hand, the accounting department may group the products differently into categories and product departments. So in this case, the product dimension table will have the attributes of marketing–product–category, marketing–product–department, finance–product–category, and finance–product–department.

Fewer Records. A dimension table typically has fewer records or rows than the fact table. A product dimension table for an automaker may have just 500 rows. On the other hand, the fact table may contain millions of rows.

Inside the Fact Table

Let us now get into a fact table and examine the components. Remember this is where we keep the measurements. We may keep the details at the lowest possible level. In the department store fact table for sales analysis, we may keep the units sold by individual transactions at the cashier’s checkout. Some fact tables may just contain summary data. These are called aggregate fact tables. Figure 10-11 lists the characteristics of a fact table. Let us review these characteristics.

Concatenated Key. A row in the fact table relates to a combination of rows from all the dimension tables. In this example of a fact table, you find quantity ordered as an attribute. Let us say the dimension tables are product, time, customer, and sales representative. For these dimension tables, assume that the lowest level in the dimension hierarchies are individual product, a calendar date, a specific customer, and a single sales representative. Then a single row in the fact table must relate to a particular product, a specific calendar date, a specific customer, and an individual sales representative. This means the row in the fact table must be identified by the primary keys of these four dimension tables. Thus, the primary key of the fact table must be the concatenation of the primary keys of all the dimension tables.

Data Grain. This is an important characteristic of the fact table. As we know, the data grain is the level of detail for the measurements or metrics. In this example, the metrics are at the detailed level. The quantity ordered relates to the quantity of a particular product on a single order, on a certain date, for a specific customer, and procured by a

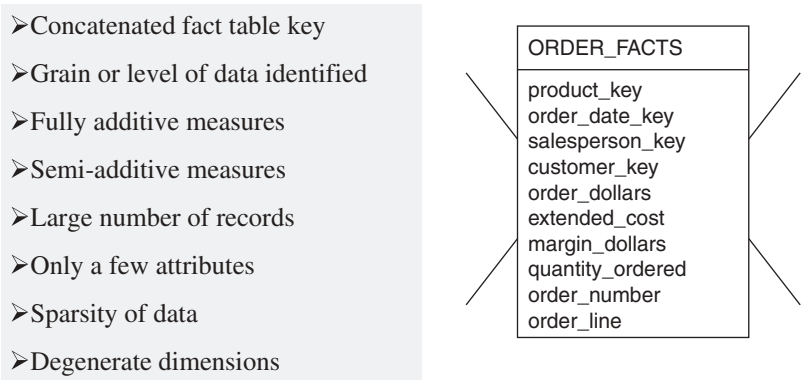


Figure 10-11 Inside a fact table.

specific sales representative. If we keep the quantity ordered as the quantity of a specific product for each month, then the data grain is different and is at a higher level.

Fully Additive Measures. Let us look at the attributes *order_dollars*, *extended_cost*, and *quantity_ordered*. Each of these relates to a particular product on a certain date for a specific customer procured by an individual sales representative. In a certain query, let us say that the user wants the totals for the particular product on a certain date, not for a specific customer, but for customers in a particular state. Then we need to find all the rows in the fact table relating to all the customers in that state and add the *order_dollars*, *extended_cost*, and *quantity_ordered* to come up with the totals. The values of these attributes may be summed up by simple addition. Such measures are known as fully additive measures. Aggregation of fully additive measures is done by simple addition. When we run queries to aggregate measures in the fact table, we will have to make sure that these measures are fully additive. Otherwise, the aggregated numbers may not show the correct totals.

Semiadditive Measures. Consider the *margin_dollars* attribute in the fact table. For example, if the *order_dollars* is 120 and *extended_cost* is 100, the *margin_percentage* is 20. This is a calculated metric derived from the *order_dollars* and *extended_cost*. If you are aggregating the numbers from rows in the fact table relating to all the customers in a particular state, you cannot add up the *margin_percentages* from all these rows and come up with the aggregated number. Derived attributes such as *margin_percentage* are not additive. They are known as semiadditive measures. Distinguish semiadditive measures from fully additive measures when you perform aggregations in queries.

Table Deep, Not Wide. Typically a fact table contains fewer attributes than a dimension table. Usually, there are about 10 attributes or less. But the number of records in a fact table is very large in comparison. Take a very simplistic example of 3 products, 5 customers, 30 days, and 10 sales representatives represented as rows in the dimension tables. Even in this example, the number of fact table rows will be 4500, very large in comparison with the dimension table rows. If you lay the fact table out as a two-dimensional table, you will note that the fact table is narrow, with a small number of columns, but very deep, with a large number of rows.

Sparse Data. We have said that a single row in the fact table relates to a particular product, a specific calendar date, a specific customer, and an individual sales representative. In other words, for a particular product, a specific calendar date, a specific customer, and an individual sales representative, there is a corresponding row in the fact table. What happens when the date represents a closed holiday and no orders are received and processed? The fact table rows for such dates will not have values for the measures. Also, there could be other combinations of dimension table attributes, values for which the fact table rows will have null measures. Do we need to keep such rows with null measures in the fact table? There is no need for this. Therefore, it is important to realize this type of sparse data and understand that the fact table could have gaps.

Degenerate Dimensions. Look closely at the example of the fact table. You find the attributes of *order_number* and *order_line*. These are not measures or metrics or facts. Then why are these attributes in the fact table? When you pick up attributes for the dimension tables and the fact tables from operational systems, you will be left with some data elements in the operational systems that are neither facts nor strictly dimension attributes. Examples of such attributes are reference numbers like order numbers, invoice numbers, order line numbers, and so on. These attributes are useful in some

types of analyses. For example, you may be looking for average number of products per order. Then you will have to relate the products to the order number to calculate the average. Attributes such as *order_number* and *order_line* in the example are called degenerate dimensions and these are kept as attributes of the fact table.

The Factless Fact Table

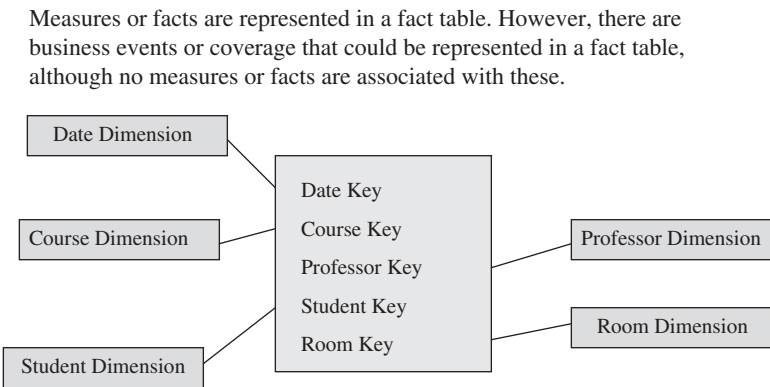
Apart from the concatenated primary key, a fact table contains facts or measures. Let us say we are building a fact table to track the attendance of students. For analyzing student attendance, the possible dimensions are student, course, date, room, and professor. The attendance may be affected by any of these dimensions. When you want to mark the attendance relating to a particular course, date, room, and professor, what is the measurement you come up with for recording the event? In the fact table row, the attendance will be indicated with the number *one*. Every fact table row will contain the number *one* as attendance. If so, why bother to record the number *one* in every fact table row? There is no need to do this. The very presence of a corresponding fact table row could indicate the attendance. This type of situation arises when the fact table represents events. Such fact tables really do not need to contain facts. They are *factless* fact tables. Figure 10-12 shows a typical factless fact table.

Data Granularity

By now, we know that granularity represents the level of detail in the fact table. If the fact table is at the lowest grain, then the facts or metrics are at the lowest possible level at which they could be captured from the operational systems. What are the advantages of keeping the fact table at the lowest grain? What is the trade-off?

When you keep the fact table at the lowest grain, users can drill down to the lowest level of detail from the data warehouse without the need to go to the operational systems themselves. Base level fact tables must be at the natural lowest levels of all corresponding dimensions. By doing this, queries for drill down and roll up can be performed efficiently.

What then are the natural lowest levels of the corresponding dimensions? In the example with the dimensions of product, date, customer, and sales representative, the natural lowest levels are an individual product, a specific individual date, an individual customer, and an



Tracks the attendance although no measured facts in the fact table

Figure 10-12 A factless fact table.

individual sales representative, respectively. So, in this case, a single row in the fact table should contain measurements at the lowest level for an individual product, ordered on a specific date, relating to an individual customer, and procured by an individual sales representative.

Let us say we want to add a new attribute of district in the sales representative dimension. This change will not warrant any changes in the fact table rows because these are already at the lowest level of individual sales representative. This is a “graceful” change because all the old queries will continue to run without any changes. Similarly, let us assume we want to add a new dimension of promotion. Now you will have to recast the fact table rows to include promotion dimensions. Still, the fact table grain will be at the lowest level.

Even here, the old queries will still run without any changes. This is also a “graceful” change. Fact tables at the lowest grain facilitate “graceful” extensions.

But we have to pay the price in terms of storage and maintenance for the fact table at the lowest grain. Lowest grain necessarily means large numbers of fact table rows. In practice, however, we build aggregate fact tables to support queries looking for summary numbers.

There are two more advantages of granular fact tables. Granular fact tables serve as natural destinations for current operational data that may be extracted frequently from operational systems. Further, the more recent data mining applications need details at the lowest grain. Data warehouses feed data into data mining applications.

STAR SCHEMA KEYS

Figure 10-13 illustrates how the keys are formed for the dimension and fact tables.

Primary Keys

Each row in a dimension table is identified by a unique value of an attribute designated as the primary key of the dimension. In a product dimension table, the primary key identifies each product uniquely. In the customer dimension table, the customer number identifies each

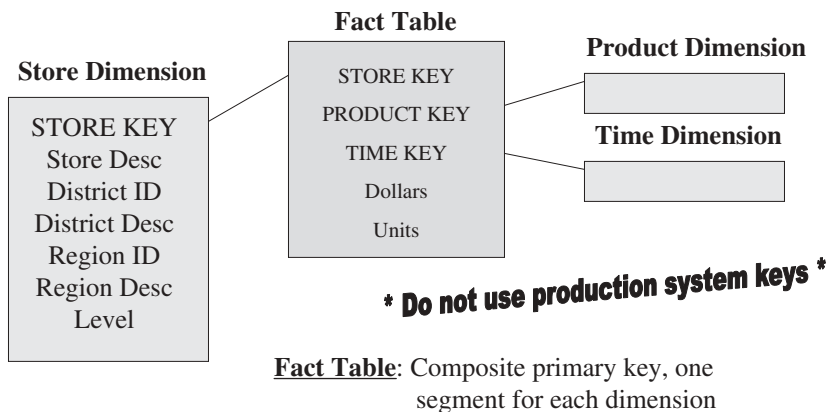


Figure 10-13 The STAR schema keys.

customer uniquely. Similarly, in the sales representative dimension table, the social security number of the sales representative identifies each sales representative.

We have picked these out as possible candidate keys for the dimension tables. Now let us consider some implications of these candidate keys. Let us assume that the product code in the operational system is an 8-position code, two of which positions indicate the code of the warehouse where the product is normally stored, and two other positions denote the product category. Let us see what happens if we use the operational system product code as the primary key for the product dimension table.

The data warehouse contains historic data. Assume that the product code gets changed in the middle of a year, because the product is now stored in a different warehouse of the company. So we have to change the product code in the data warehouse. If the product code is the primary key of the product dimension table, then the newer data for the same product will reside in the data warehouse with different key values. This could cause problems if we need to aggregate the data from before the change with the data from after the change to the product code. What really has caused this problem? The problem is the result of our decision to use the operational system key as the key for the dimension table.

Surrogate Keys

How do we resolve the problem faced in the previous section? Can we use production system keys as primary keys for dimension tables? If not, what are the other candidate keys?

There are two general principles to be applied when choosing primary keys for dimension tables. The first principle is derived from the problem caused when the product began to be stored in a different warehouse. In other words, the product key in the operational system has built-in meanings. Some positions in the operational system product key indicate the warehouse and some other positions in the key indicate the product category. These are built-in meanings in the key. The first principle to follow is: avoid built-in meanings in the primary key of the dimension tables.

In some companies, a few of the customers are no longer listed with the companies. They could have left their respective companies many years ago. It is possible that the customer numbers of such discontinued customers are reassigned to new customers. Now, let us say we had used the operational system customer key as the primary key for the customer dimension table. We will have a problem because the same customer number could relate to the data for the newer customer and also to the data of the retired customer. The data of the retired customer may still be used for aggregations and comparisons by city and state. Therefore, the second principle is: do not use production system keys as primary keys for dimension tables.

What then should we use as primary keys for dimension tables? The answer is to use surrogate keys. The surrogate keys are simply system-generated sequence numbers. They do not have any built-in meanings. Of course, the surrogate keys will be mapped to the production system keys. Nevertheless, they are different. The general practice is to keep the operational system keys as additional attributes in the dimension tables. Refer back to Figure 10-13. The STORE KEY is the surrogate primary key for the store dimension table. The operational system primary key for the store reference table may be kept as just another non-key attribute in the store dimension table.

Foreign Keys

Each dimension table is in a one-to-many relationship with the central fact table. So the primary key of each dimension table must be a foreign key in the fact table. If there are

four dimension tables of product, date, customer, and sales representative, then the primary key of each of these four tables must be present in the orders fact table as foreign keys.

Let us reexamine the primary keys for the fact tables. There are three options:

1. *A single compound primary key whose length is the total length of the keys of the individual dimension tables.* Under this option, in addition to the compound primary key, the foreign keys must also be kept in the fact table as additional attributes. This option increases the size of the fact table.
2. *A concatenated primary key that is the concatenation of all the primary keys of the dimension tables.* Here you need not keep the primary keys of the dimension tables as additional attributes to serve as foreign keys. The individual parts of the primary keys themselves will serve as the foreign keys.
3. *A generated primary key independent of the keys of the dimension tables.* In addition to the generated primary key, the foreign keys must also be kept in the fact table as additional attributes. This option also increases the size of the fact table.

In practice, option (2) is used in most fact tables. This option enables you to easily relate the fact table rows with the dimension table rows.

ADVANTAGES OF THE STAR SCHEMA

When you look at the STAR schema, you find that it is simply a relational model with a one-to-many relationship between each dimension table and the fact table. What is so special about the arrangement of the STAR schema? Why is it declared to be eminently suitable for the data warehouse? What are the reasons for its wide use and success in providing optimization for processing queries?

Although the STAR schema is a relational model, it is not a normalized model. The dimension tables are purposely denormalized. This is a basic difference between the STAR schema and relational schemas for OLTP systems.

Before we discuss some very significant advantages of the STAR schema, we need to be aware that strict adherence to this arrangement is not always the best option. For example, if customer is one of the dimensions and if the enterprise has a very large number of customers, a denormalized customer dimension table is not desirable. A large dimension table may increase the size of the fact table correspondingly.

However, the advantages far outweigh any shortcomings. So, let us go over the advantages of the STAR schema.

Easy for Users to Understand

Users of OLTP systems interact with the applications through predefined GUI screens or preset query templates. There is practically no need for the users to understand the data structures behind the scenes. The data structures and the database schema remain in the realm of the IT professionals.

Users of decision support systems such as data warehouses are different. Here the users themselves will formulate queries. When they interact with the data warehouse through third-party query tools, the users should know what to ask for. They must gain a familiarity with what data is available to them in the data warehouse. They must have an understanding of the data structures and how the various pieces are associated with one another in the overall scheme. They must comprehend the connections without difficulty.

The STAR schema reflects exactly how the users think and need data for querying and analysis. They think in terms of significant business metrics. The fact table contains the metrics. The users think in terms of business dimensions for analyzing the metrics. The dimension tables contain the attributes along which the users normally query and analyze. When you explain to the users that the units of product A are stored in the fact table and point out the relationship of this piece of data to each dimension table, the users readily understand the connections. That is because the STAR schema defines the join paths in exactly the same way users normally visualize the relationships. The STAR schema is intuitively understood by the users.

Try to walk a user through the relational schema of an OLTP system. For them to understand the connections, you will have to take them through a maze of normalized tables, sometimes passing through several tables, one by one, to get even the smallest result set. The STAR schema emerges as a clear winner because of its simplicity. Users understand the structures and the connections very easily.

The STAR schema has definite advantages after implementation. However, the advantages even in the development stage cannot be overlooked. Because the users understand the STAR schema so very well, it is easy to use it as a vehicle for communicating with the users during the development of the data warehouse.

Optimizes Navigation

In a database schema, what is the purpose of the relationships or connections among the data entities? The relationships are used to go from one table to another for obtaining the information you are looking for. The relationships provide the ability to navigate through the database. You hop from table to table using the join paths.

If the join paths are numerous and convoluted, your navigation through the database gets difficult and slow. On the other hand, if the join paths are simple and straightforward, your navigation is optimized and becomes faster.

A major advantage of the STAR schema is that it optimizes the navigation through the database. Even when you are looking for a query result that is seemingly complex, the navigation is still simple and straightforward. Let us look at an example and understand how this works. Look at Figure 10-14 showing a STAR schema for analyzing defects in automobiles. Assume you are the service manager at an automobile dealership selling GM automobiles. You noticed a high incidence of chipped white paint on the Corvettes in January 2009. You need a tool to analyze such defects, determine the underlying causes, and resolve the problems.

In the STAR schema, the number of defects is kept as metrics in the middle as part of the defects fact table. The time dimension contains the model year. The component dimension has part information; for example, pearl white paint. The problem dimension carries the types of problems; for example, chipped paint. The product dimension contains the make, model, and trim package of the automobiles. The supplier dimension contains data on the suppliers of parts.

Now see how easy it is to determine the supplier causing the chipped paint on the pearl white Corvettes. Look at the four arrows pointing to the fact table from the four dimension tables. These arrows show how you will navigate to the rows in the fact table by isolating the Corvette from the product dimension, chipped paint from the problem dimension, pearl white paint from the component dimension, and January 2009 from the time dimension. From the fact table, the navigation goes directly to the supplier dimension to isolate the supplier causing the problem.

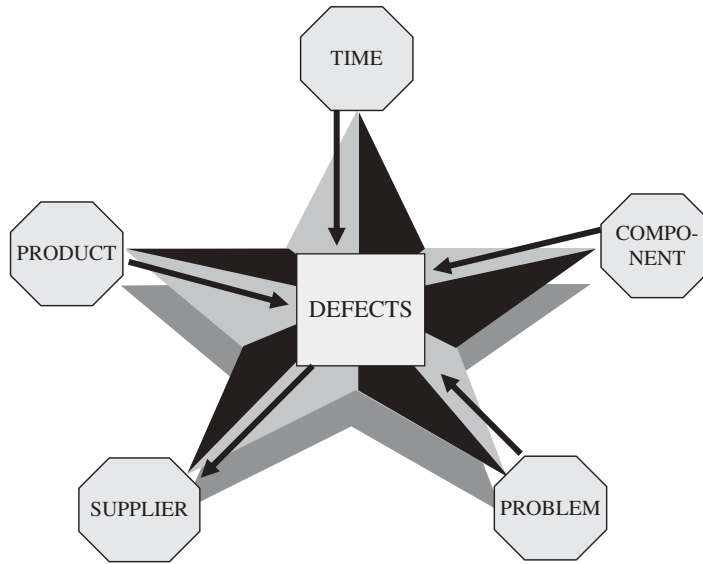


Figure 10-14 The STAR schema optimizes navigation.

Most Suitable for Query Processing

We have already mentioned a few times that the STAR schema is a query-centric structure. This means that the STAR schema is most suitable for query processing. Let us see how this is true.

Let us form a simple query on the STAR schema for the order analysis shown in Figure 10-7. What is the total extended cost of product A sold to customers in San Francisco during January 2009? This is a three-dimensional query. What should be the characteristics of the data structure or schema if it is to be most suitable for processing this query? The final result, which is the total extended cost, will come from the rows in the fact table. But from which rows? The answer is those rows relating to product A, relating to customers in San Francisco, and relating to January 2009.

Let us see how the query will be processed. First, select the rows from the customer dimension table where the city is San Francisco. Then, from the fact table, select only those rows that are related to these customer dimension rows. This is the first result set of rows from the fact tables. Next, select the rows in the Time dimension table where the month is January 2009. Select from the first result set only those rows that are related to these time dimension rows. This is now the second result set of fact table rows. Move on to the next dimension of product. Select the rows in the product dimension table where the product is product A. Select from the second result only those rows that are related to the selected product dimension rows. You now have the final result of fact table rows. Add up the extended cost to get the total.

Irrespective of the number of dimensions that participate in the query and irrespective of the complexity of the query, every query is simply executed first by selecting rows from the dimension tables using the filters based on the query parameters and then finding the corresponding fact table rows. This is possible because of the simple and straightforward join paths and because of the very arrangement of the STAR schema. There is no intermediary maze to be navigated to reach the fact table from the dimension tables.

Another important aspect of data warehouse queries is the ability to drill down or roll up. Let us quickly run through a drill-down scenario. Let us say we have queried and obtained the total extended cost for all the customers in the state of California. The result comes from the set of fact table rows. Then we want to drill down and look at the results by zip code ranges. This is obtained by making a further selection from the selected fact table rows relating to the chosen zip code ranges. Drill down is a process of further selection of the fact table rows. Going the other way, rolling up is a process of expanding the selection of the fact table rows.

STARjoin and STARindex

The STAR schema allows the query processor software to use better execution plans. It enables specific performance schemes to be applied to queries. The STAR schema arrangement is eminently suitable for special performance techniques such as the STARjoin and the STARindex.

STARjoin is a high-speed, single-pass, parallelizable, multitable join. It can join more than two tables in a single operation. This special scheme boosts query performance.

STARindex is a specialized index to accelerate join performance. These are indexes created on one or more foreign keys of the fact table. These indexes speed up joins between the dimension tables and the fact table.

Chapter 18 deals with the physical design and indexing methods for the data warehouse.

STAR SCHEMA: EXAMPLES

In this section we provide four examples of STAR schema relating to four different types of businesses. In each case, notice the following carefully and understand the STAR schema:

- The metrics or facts being analyzed
- The business dimensions used for analysis
- The hierarchy available within each dimension for drill down and roll up
- The primary and foreign keys

Video Rental

See Figure 10-15.

Supermarket

See Figure 10-16.

Wireless Phone Service

See Figure 10-17.

Auction Company

See Figure 10-18.

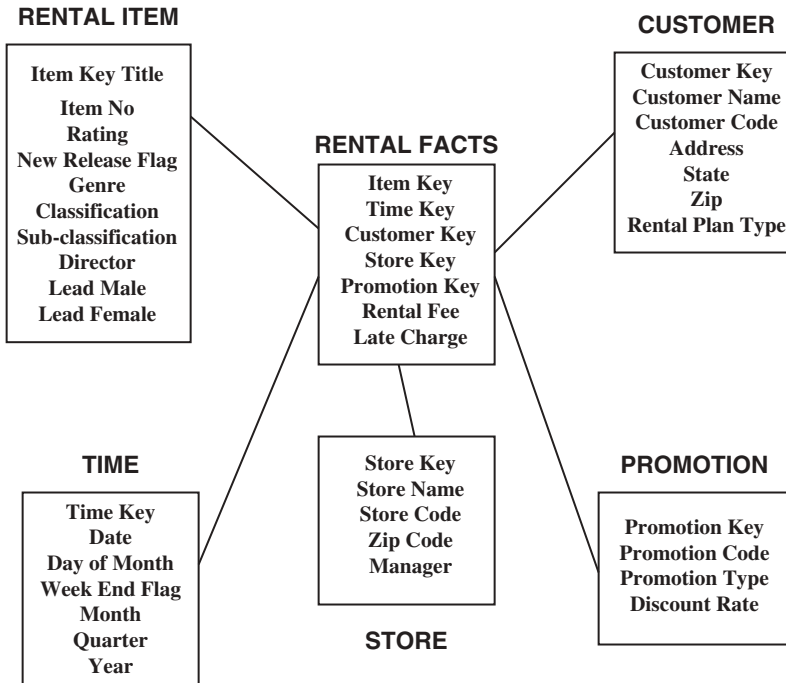


Figure 10-15 STAR schema example: video rental.

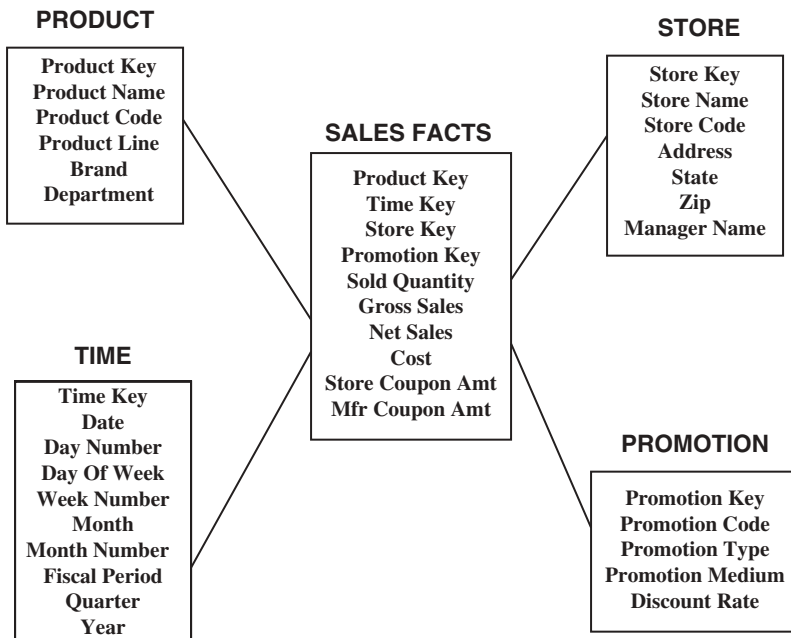


Figure 10-16 STAR schema example: supermarket.

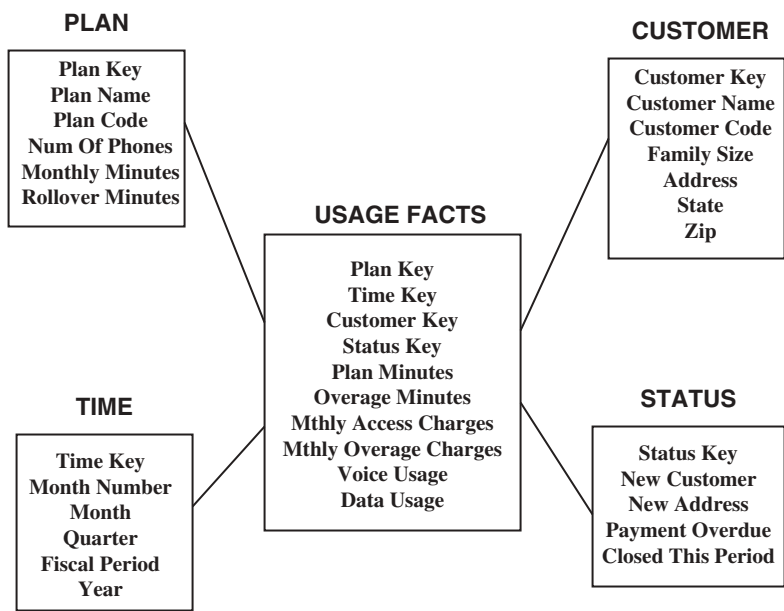


Figure 10-17 STAR schema example: wireless phone service.

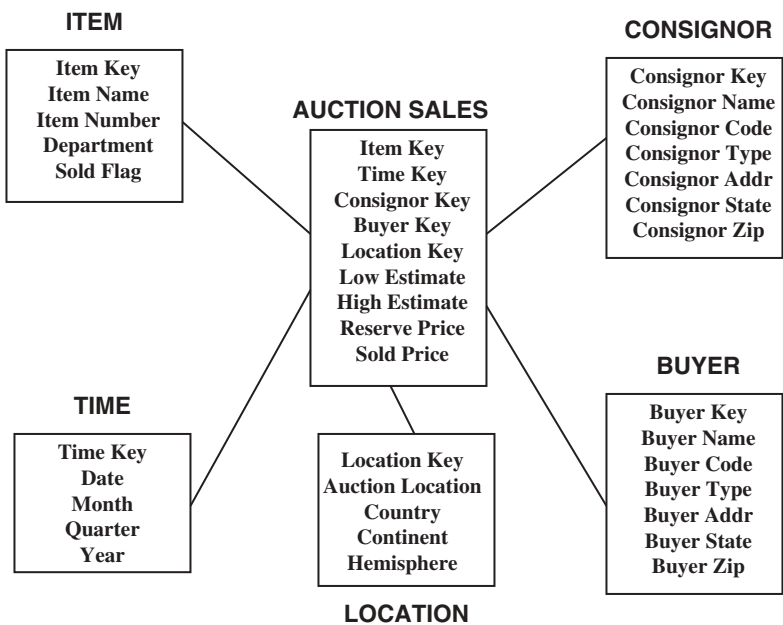


Figure 10-18 STAR schema example: auction company.

CHAPTER SUMMARY

- The components of the dimensional model are derived from the information packages in the requirements definition.

- The entity-relationship modeling technique is not suitable for data warehouses; the dimensional modeling technique is appropriate.
- The STAR schema used for data design is a relational model consisting of fact and dimension tables.
- The fact table contains the business metrics or measurements; the dimension tables contain the business dimensions. Hierarchies within each dimension table are used for drilling down to lower levels of data.
- STAR schema advantages are that it is easy for users to understand optimizes navigation, is most suitable for query processing, and enables specific performance schemes.

REVIEW QUESTIONS

1. Discuss the major design issues that need to be addressed before proceeding with the data design.
2. Why is the entity-relationship modeling technique not suitable for the data warehouse? How is dimensional modeling different?
3. What is the STAR schema? What are the component tables?
4. A dimension table is wide; the fact table is deep. Explain.
5. What are hierarchies and categories as applicable to a dimension table?
6. Differentiate between fully additive and semiadditive measures.
7. Explain the sparse nature of the data in the fact table.
8. Describe the composition of the primary keys for the dimension and fact tables.
9. Discuss data granularity in a data warehouse.
10. Name any three advantages of the STAR schema. Can you think of any disadvantages of the STAR schema?

EXERCISES

1. Match the columns:

- | | |
|-----------------------------|----------------------------------|
| 1. information package | A. enable drill down |
| 2. fact table | B. reference numbers |
| 3. case tools | C. level of detail |
| 4. dimension hierarchies | D. users understand easily |
| 5. dimension table | E. semiadditive |
| 6. degenerate dimensions | F. STAR schema components |
| 7. profit margin percentage | G. used for dimensional modeling |
| 8. data granularity | H. dimension attribute |
| 9. STAR schema | I. contains metrics |
| 10. customer demographics | J. wide |

2. Refer back to the information package given for a hotel chain in Chapter 5 (Fig. 5.6). Use this information package and design a STAR schema.
3. What is a factless fact table? Design a simple STAR schema with a factless fact table to track patients in a hospital by diagnostic procedures and time.
4. You are the data design specialist on the data warehouse project team for a manufacturing company. Design a STAR schema to track the production quantities. Production quantities are normally analyzed along the business dimensions of product, time, parts used, production facility, and production run. State your assumptions.
5. In a STAR schema to track the shipments for a distribution company, the following dimension tables are found: (1) time, (2) customer ship-to, (3) ship-from, (4) product, (5) type of deal, and (6) mode of shipment. Review these dimensions and list the possible attributes for each of the dimension tables. Also, designate a primary key for each table.

CHAPTER 11

DIMENSIONAL MODELING: ADVANCED TOPICS

CHAPTER OBJECTIVES

- Discuss and get a good grasp of slowly changing dimensions
- Understand large dimensions and how to deal with them
- Examine the snowflake schema in detail
- Learn about aggregate tables and determine when to use them
- Completely survey families of STARS and their applications

From the previous chapter, you have learned the basics of dimensional modeling. You know that the STAR schema is composed of the fact table in the middle surrounded by the dimension tables. Although this is a good visual representation, it is still a relational model in which each dimension table is in a parent-child relationship with the fact table. The primary key of each dimension table, therefore, is a foreign key in the fact table.

You have also grasped the nature of the attributes within the fact table and the dimension tables. You have understood the advantages of the STAR schema in decision support systems. The STAR schema is easy for users to understand; it optimizes navigation through the data warehouse content and is most suitable for query-centric environments.

Our study of dimensional modeling will not be complete until we consider some more topics. In the STAR schema, the dimension tables enable analysis in many different ways. We need to explore the dimension tables in further detail. How about summarizing the metrics and storing aggregate numbers in additional fact tables? How much precalculated aggregation is necessary? The STAR schema is a denormalized design. Does this result in too much redundancy and inefficiency? If so, is there an alternative approach?

Let us now move beyond the basics of dimensional modeling and consider additional features and issues. Let us discuss the pertinent advanced topics and extend our study further.

UPDATES TO THE DIMENSION TABLES

Going back to Figure 10-4 of the previous chapter, you see the STAR schema for automaker sales. The fact table *Auto Sales* contains the measurements or metrics such as *Actual Sale Price*, *Options Price*, and so on. Over time, what happens to the fact table? Every day as more and more sales take place, more and more rows get added to the fact table. Over time the number of rows in the fact table continues to grow. Very rarely are the rows in a fact table updated with changes. Even when there are adjustments to the prior numbers, these are also processed as additional adjustment rows and added to the fact table.

Now consider the dimension tables. Compared to the fact table, the dimension tables are more stable and less volatile. However, unlike the fact table, which changes through an increase in the number of rows, a dimension table does not change just through the increase in the number of rows, but also through changes to the attributes themselves.

Look at the product dimension table. Every year, rows are added as new models become available. But what about the attributes within the product dimension table? If a particular product is moved to a different product category, then the corresponding values must be changed in the product dimension table. Let us examine the types of changes that affect dimension tables and discuss the ways for dealing with these types.

Slowly Changing Dimensions

In the above example, we mentioned a change to the product dimension table because the product category for a product was changed. Consider the customer demographics dimension table. What happens when a customer's status changes from rental home to own home? The corresponding row in that dimension table must be changed. Next, look at the payment method dimension table. When finance type changes for one of the payment methods, this change must be reflected in the payment method dimension table.

From the consideration of the changes to the dimension tables, we can derive the following principles:

- Most dimensions are generally constant over time.
- Many dimensions, though not constant over time, change slowly.
- The product key of the source record does not change.
- The description and other attributes change slowly over time.
- In the source OLTP systems, the new values overwrite the old ones.
- Overwriting of dimension table attributes is not always the appropriate option in a data warehouse.
- The ways changes are made to the dimension tables depend on the types of changes and what information must be preserved in the data warehouse.

Techniques for making the changes fall into three distinct types. Data warehousing practitioners have come up with different techniques for applying the changes. They have also given names to these three types of dimension table changes. Yes, your guess is right. The given names are type 1 changes, type 2 changes, and type 3 changes.

We will study these three types by using a simple STAR schema for tracking orders for a distributor of industrial products, as shown in Figure 11-1. This STAR schema consists of

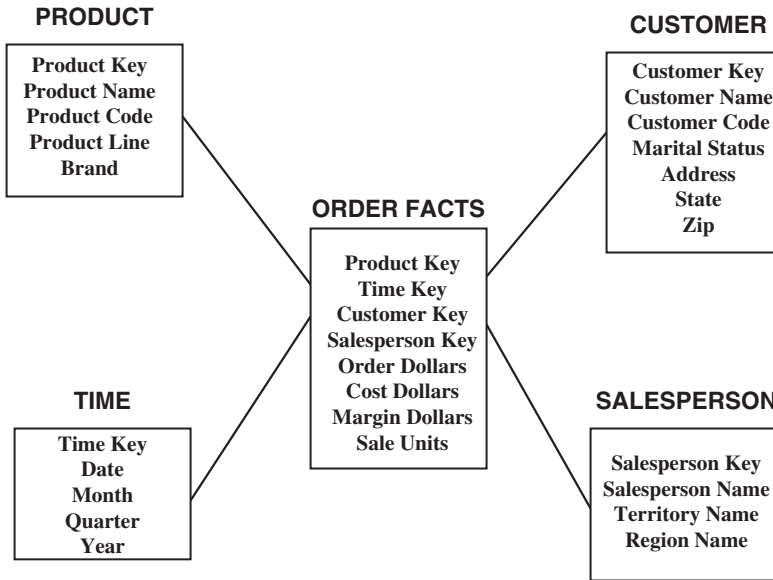


Figure 11-1 STAR schema for order tracking.

the fact table and four dimension tables. Let us assume some changes to these dimensions and review the techniques for applying the changes to the dimension tables.

Type 1 Changes: Correction of Errors

Nature of Type 1 Changes These changes usually relate to the corrections of errors in the source systems. For example, suppose a spelling error in the customer name is corrected to read Michael Romano from the erroneous entry of Michel Romano. Also, suppose the customer name for another customer is changed from Kristin Daniels to Kristin Samuelson, and the marital status changed from single to married.

Consider the changes to the customer name in both cases. There is no need to preserve the old values. In the case of Michael Romano, the old name is erroneous and needs to be discarded. When a user needs to find all the orders from Michael Romano, he or she will use the correct name. The same principles apply to the change in customer name for Kristin Samuelson.

But the change in the marital status is slightly different. This change can be handled in the same way as the change in customer name only if that change is a correction of error. Otherwise, you will cause problems when the users want to analyze orders by marital status.

Here are the general principles for type 1 changes:

- Usually, the changes relate to correction of errors in source systems.
- Sometimes the change in the source system has no significance.
- The old value in the source system needs to be discarded.
- The change in the source system need not be preserved in the data warehouse.

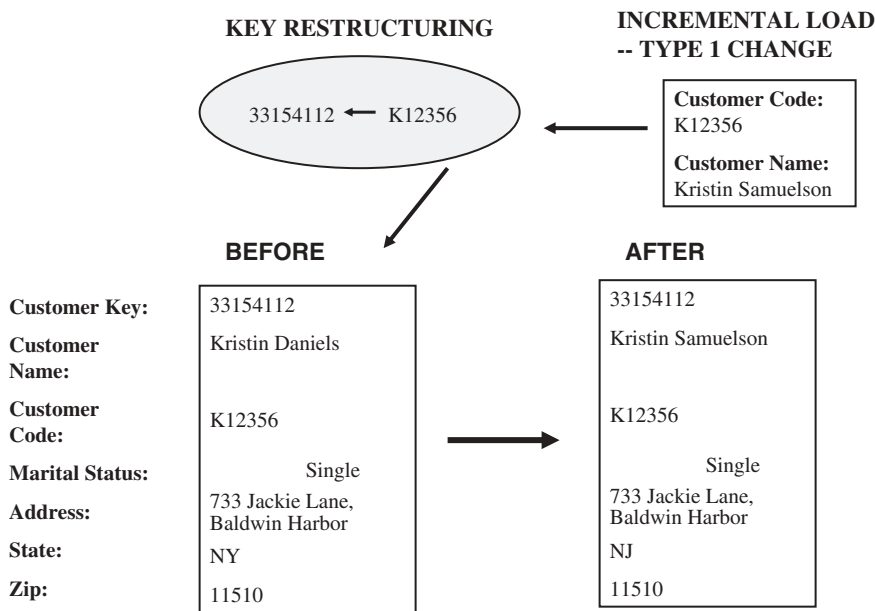


Figure 11-2 The method for applying type 1 changes.

Applying Type 1 Changes to the Data Warehouse Figure 11-2 shows the application of type 1 changes to the customer dimension table. The method for applying type I changes is:

- Overwrite the attribute value in the dimension table row with the new value.
- The old value of the attribute is not preserved.
- No other changes are made in the dimension table row.
- The key of this dimension table or any other key values are not affected.
- This type is easiest to implement.

Type 2 Changes: Preservation of History

Nature of Type 2 Changes Go back to the change in the marital status for Kristin Samuelson. Assume that in your data warehouse one of the essential requirements is to track orders by marital status in addition to tracking by other attributes. If the change to marital status happened on October 1, 2008, all orders from Kristin Samuelson before that date must be included under marital status: single, and all orders on or after October 1, 2008 should be included under marital status: married.

What exactly is needed in this case? In the data warehouse, you must have a way of separating the orders for the customer so that the orders before and after that date can be added up separately.

Now let us add another change to the information about Kristin Samuelson. Assume that she moved to a new address in California from her old address in New York on November 1, 2008. If it is a requirement in your data warehouse that you must be able to track orders by state, then this change must also be treated like the change to marital status. Any orders prior to November 1, 2008 will go under the state: NY.

The types of changes we have discussed for marital status and customer address are type 2 changes. Here are the general principles for this type of change:

- They usually relate to true changes in source systems.
- There is a need to preserve history in the data warehouse.
- This type of change partitions the history in the data warehouse.
- Every change for the same attribute must be preserved.

Applying Type 2 Changes to the Data Warehouse Figure 11-3 shows the application of type 2 changes to the customer dimension table. The method for applying type 2 changes is:

- Add a new dimension table row with the new value of the changed attribute.
- An effective date field may be included in the dimension table.
- There are no changes to the original row in the dimension table.
- The key of the original row is not affected.
- The new row is inserted with a new surrogate key.

Type 3 Changes: Tentative Soft Revisions

Nature of Type 3 Changes Almost all the usual changes to dimension values are either type 1 or type 2 changes. Of these two, type 1 changes are more common. Type 2 changes preserve the history. When you apply a type 2 change on a certain date, that date

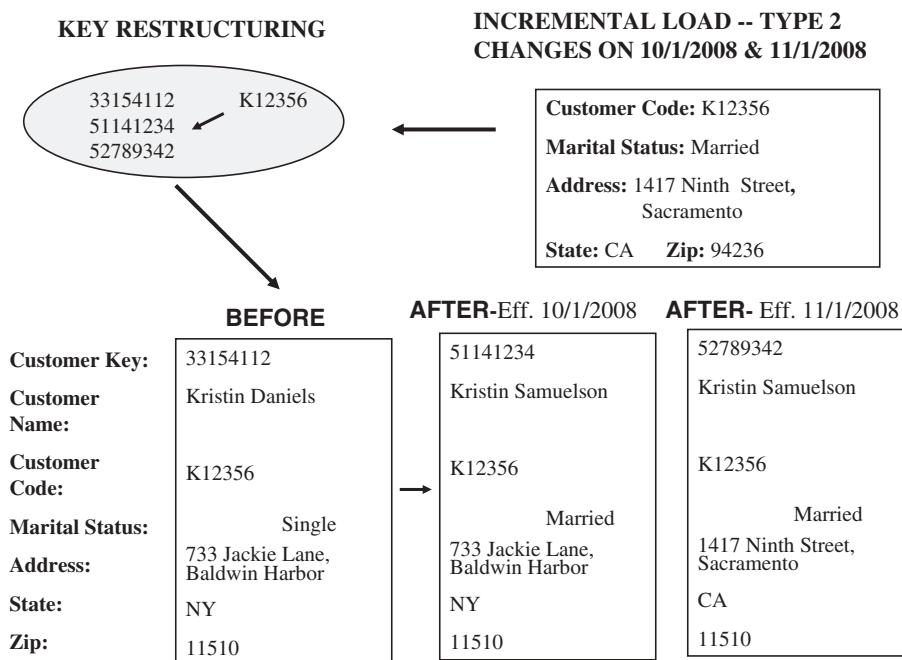


Figure 11-3 The method for applying type 2 changes.

is a cut-off point. In the above case of change to marital status on October 1, 2008, that date is the cut-off date. Any orders from the customer prior to that date fall into the older orders group; orders on or after that date fall into the newer orders group. An order for this customer has to fall in one or the other group; it cannot be counted in both groups for any period of time.

What if you have the need to count the orders on or after the cut-off date in both groups during a certain period after the cut-off date? You cannot handle this change as a type 2 change. Sometimes, though rarely, there is a need to track both the old and new values of changed attributes for a certain period, in both forward and backward directions. These types of changes are type 3 changes.

Type 3 changes are tentative or soft changes. An example will make this clearer. Assume your marketing department is contemplating a realignment of the territorial assignments for salespersons. Before making a permanent realignment, they want to count the orders in two ways: according to the current territorial alignment and also according to the proposed realignment. This type of provisional or tentative change is a type 3 change.

As an example, let us say you want to move salesperson Robert Smith from the New England territory to the Chicago territory with the ability to trace his orders in both territories. You need to track all orders through Robert Smith in both territories.

Here are the general principles for type 3 changes:

- They usually relate to “soft” or tentative changes in the source systems.
- There is a need to keep track of history with old and new values of the changed attribute.
- They are used to compare performances across the transition.
- They provide the ability to track forward and backward.

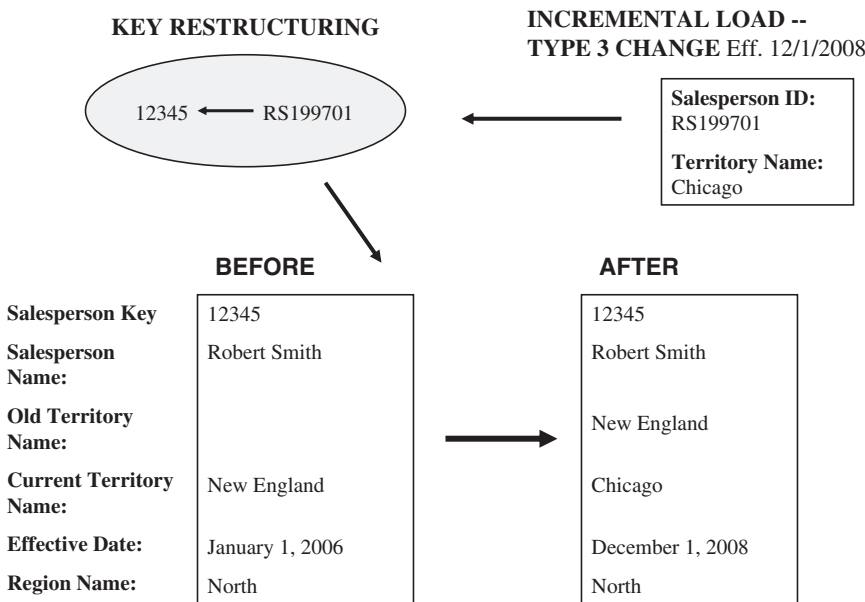


Figure 11-4 Applying type 3 changes.

Applying Type 3 Changes to the Data Warehouse Figure 11-4 shows the application of type 3 changes to the customer dimension table. The methods for applying type 3 changes are:

- Add an “old” field in the dimension table for the affected attribute.
- Push down the existing value of the attribute from the “current” field to the “old” field.
- Keep the new value of the attribute in the “current” field.
- Also, you may add a “current” effective date field for the attribute.
- The key of the row is not affected.
- No new dimension row is needed.
- The existing queries will seamlessly switch to the “current” value.
- Any queries that need to use the “old” value must be revised accordingly.
- The technique works best for one “soft” change at a time.
- If there is a succession of changes, more sophisticated techniques must be devised.

MISCELLANEOUS DIMENSIONS

Having considered the types of changes to dimension attributes and the ways to handle the dimension changes in the data warehouse, let us now turn our attention to a few other important issues about dimensions. One issue relates to dimension tables that are very wide and very deep.

In our earlier discussion, we had assumed that dimension attributes do not change too rapidly. If the change is a type 2 change, you know that you have to create another row with the new value of the attribute. If the value of the attribute changes again, then you create another row with the newer value. What if the value changes too many times or too rapidly? Such a dimension is no longer a slowly changing dimension. What must you do about a not-so-slowly-changing dimension? We will complete our discussion of dimensions by considering such relevant issues.

Large Dimensions

You may consider a dimension large based on two factors. A large dimension is very deep; that is, the dimension has a very large number of rows. A large dimension may also be very wide; that is, the dimension may have a large number of attributes or columns. In either case, you may declare the dimension large. There are special considerations for large dimensions. You may have to attend to populating large-dimension tables in a special way. You may want to separate out some mini-dimensions from a large dimension. We will take a simple STAR schema designed for order analysis. Assume this to be the schema for a manufacturing company and that the marketing department is interested in determining how they are making progress with the orders received by the company.

In a data warehouse, typically the customer and product dimensions are likely to be large. Whenever an enterprise deals with the general public, the customer dimension is expected to be gigantic. The customer dimension of a national retail chain can approach the size of the number of U.S. households. Such customer dimension tables may have as many as 100 million rows. Next on the scale, the number of dimension table rows of companies in telecommunications and travel industries may also run in the millions. Having 10 or 20

million customer rows is not uncommon. The product dimension of large retailers is also quite huge.

Here are some typical features of large customer and product dimensions:

Customer

- Huge—in the range of 20 million rows
- Easily up to 150 dimension attributes
- Can have multiple hierarchies

Product

- Sometimes as many as 100,000 product variations
- Can have more than 100 dimension attributes
- Can have multiple hierarchies

Large dimensions call for special considerations. Because of the sheer size, many data warehouse functions involving large dimensions may be slow and inefficient. You need to address the following issues by using effective design methods, by choosing proper indexes, and by applying other optimizing techniques:

- Populating very large dimension tables
- Browse performance of unconstrained dimensions, especially where the cardinality of the attributes is low
- Browsing time for cross-constrained values of the dimension attributes
- Inefficiencies in fact table queries when large dimensions need to be used
- Additional rows created to handle type 2 slowly changing dimensions

Multiple Hierarchies Large dimensions usually possess another distinct characteristic. They tend to have multiple hierarchies. Take the example of the product dimension for a large retailer. One set of attributes may form the hierarchy for the marketing department. Users from that department use these attributes to drill down or up. In the same way, the finance department may need to use its own set of attributes from the same product dimension to drill down or up. Figure 11-5 shows multiple hierarchies within a large product dimension.

Rapidly Changing Dimensions

As you know, when you deal with a type 2 change, you create an additional dimension table row with the new value of the changed attribute. By doing so, you are able to preserve the history. If the same attribute changes a second time, you create one more dimension table row with the latest value.

Most product dimensions change very infrequently, maybe once or twice a year. If the number of rows in such a product dimension is about 100,000 or so, using the approach of creating additional rows with the new values of the attributes is easily manageable. Even if the number of rows is in the range of several thousands, the approach of applying the changes as type 2 changes is still quite feasible.

However, consider another dimension, such as the customer dimension. Here the number of rows tends to be large, sometimes in the range of even a million or more rows. If the

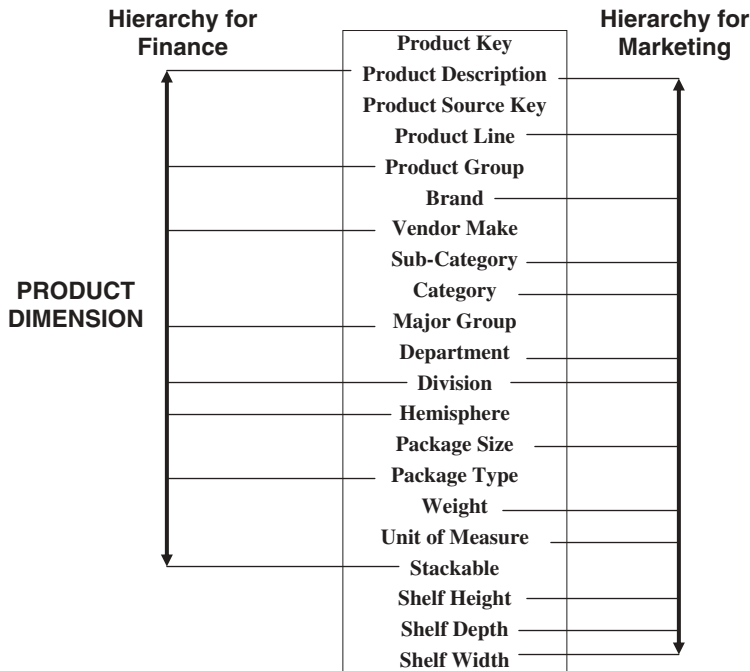


Figure 11-5 Multiple hierarchies in a large product dimension.

attributes of a large number of rows change, but change infrequently, the type 2 approach is not too difficult. But significant attributes in a customer dimension may change many times in a year. Rapidly changing large dimensions can be too problematic for the type 2 approach. The dimension table could be littered with a very large number of additional rows created every time there is an incremental load.

Before rushing to explore other options for handling rapidly changing large dimensions, deal with each large dimension individually. The type 2 approach is still good in a STAR schema design. Here are some reasons why the type 2 approach could work in many cases for rapidly changing dimensions:

- When the dimension table is kept flat, it allows symmetric cross-browsing among the various attributes of the dimension.
- Even when additional dimension table rows get created, the basic dimensional structure is still preserved. The fact table is connected to all the dimension tables by foreign keys. The advantages of the STAR schema are still available.
- Only when the end-user queries are based on a changed attribute does the existence of multiple rows for the same customer become apparent. For other queries, the existence of multiple rows is practically hidden.

What if the dimension table is too large and is changing too rapidly? Then seek alternatives to straightforward application of the type 2 approach. One effective approach is to break the large dimension table into one or more simpler dimension tables. How can you accomplish this?

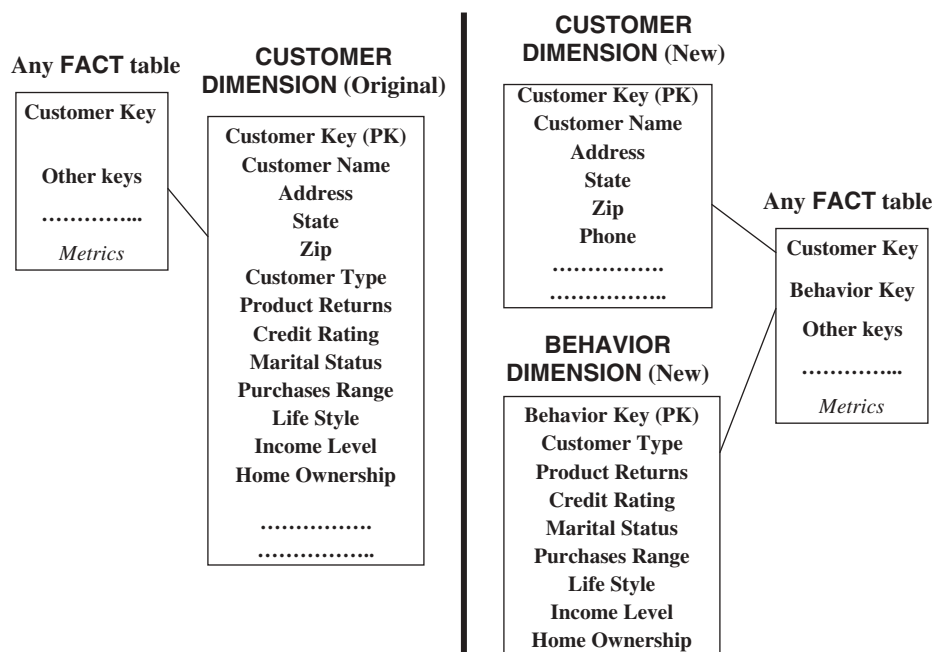


Figure 11-6 Dividing a large, rapidly changing dimension table.

Obviously, you need to break off the rapidly changing attributes into another dimension table, leaving the slowly changing attributes behind in the original table. Figure 11-6 shows how a customer dimension table may be separated into two dimension tables. The figure illustrates the general technique of separating out the rapidly changing attributes. Use this as a guide when dealing with large, rapidly changing dimensions in your data warehouse environment.

Junk Dimensions

Examine your source legacy systems and review the individual fields in source data structures for customer, product, order, sales territories, promotional campaigns, and so on. Most of these fields wind up in the dimension tables. You will notice that some fields like miscellaneous flags and textual fields are left in the source data structures. These include yes/no flags, textual codes, and free form texts.

A few of these flags and textual data may be too obscure to be of real value. These may be leftovers from past conversions from manual records created long ago. However, many of the flags and texts could be of value once in a while in queries. These may not be included as significant fields in the major dimensions. At the same time, these flags and texts cannot be discarded either. So, what are your options? Here are the main choices:

- Exclude and discard all flags and texts. Obviously, this is not a good option, for the simple reason that you are likely to throw away some useful information.
- Place the flags and texts unchanged in the fact table. This option is likely to swell up the fact table to no specific advantage.

- Make each flag and text a separate dimension table on its own. Using this option, the number of dimension tables will greatly increase.
- Keep only those flags and texts that are meaningful; group all the useful flags into a single “junk” dimension. “Junk” dimension attributes are useful for constraining queries based on flag/text values.

THE SNOWFLAKE SCHEMA

“Snowflaking” is a method of normalizing the dimension tables in a STAR schema. When you completely normalize all the dimension tables, the resultant structure resembles a snowflake with the fact table in the middle. First, let us begin with Figure 11-7, which shows a simple STAR schema for sales in a manufacturing company.

The sales fact table contains quantity, price, and other relevant metrics. Sales rep, customer, product, and time are the dimension tables. This is a classic STAR schema, denormalized for optimal query access involving all or most of the dimensions. The model is not in the third normal form.

Options to Normalize

Assume that there are 500,000 product dimension rows. These products fall under 500 product brands and these product brands fall under 10 product categories. Now suppose one of your users runs a query constraining just on product category. If the product dimension table is not indexed on product category, the query will have to search through 500,000 rows. On the other hand, even if the product dimension is partially normalized by separating out product brand and product category into separate tables, the initial search for the query will have to go through only 10 rows in the product category table. Figure 11-8 illustrates this reduction in the search process.

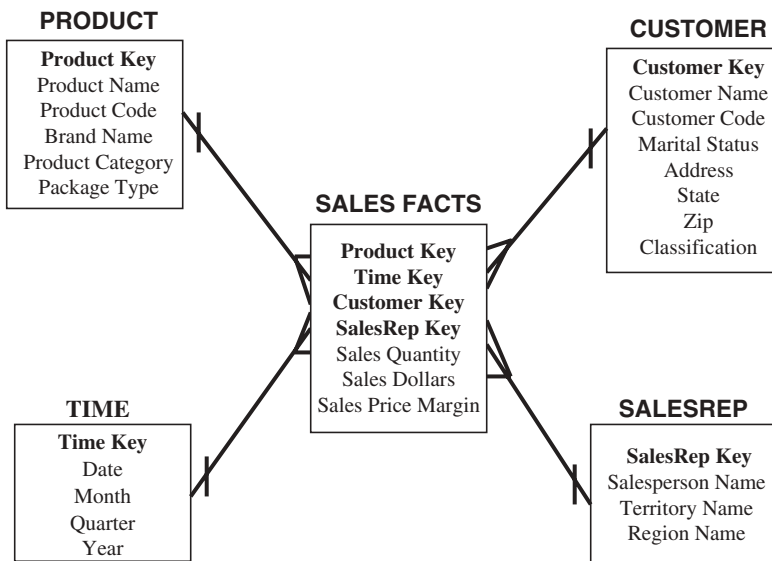


Figure 11-7 Sales: a simple STAR schema.

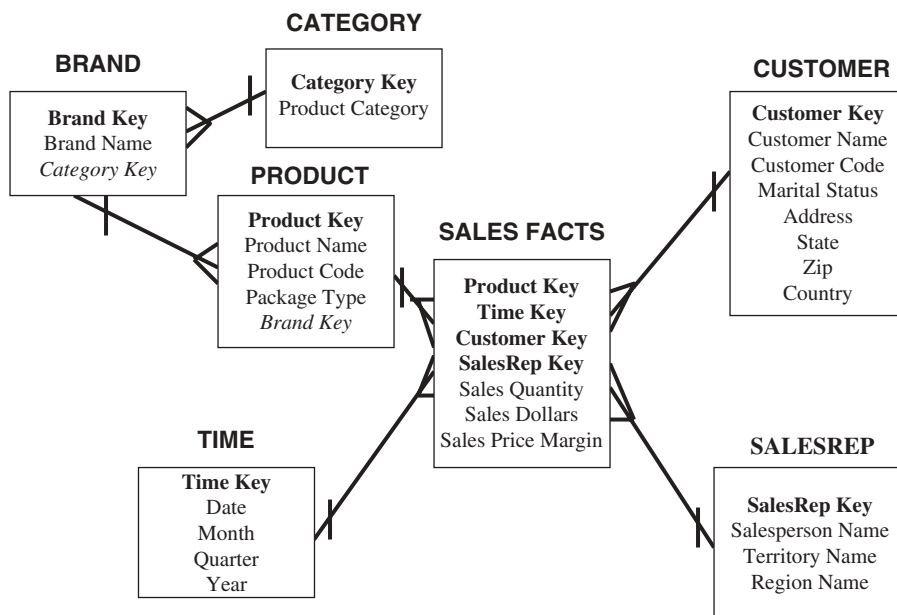


Figure 11-8 Product dimension: partially normalized.

In Figure 11-8, we have not completely normalized the product dimension. We can also move other attributes out of the product dimension table and form normalized structures. “Snowflaking” or normalization of the dimension tables can be achieved in a few different ways. When you want to “snowflake,” examine the contents and the normal usage of each dimension table.

The following options indicate the different methods you may want to consider for normalization of the dimension tables:

- Partially normalize only a few dimension tables, leaving the others intact.
- Partially or fully normalize only a few dimension tables, leaving the rest intact.
- Partially normalize every dimension table.
- Fully normalize every dimension table.

Figure 11-9 shows the version of the snowflake schema for sales in which every dimension table is partially or fully normalized.

The original STAR schema for sales as shown in Figure 11-7 contains only 5 tables, whereas the normalized version now extends to 11 tables. You will notice that in the snowflake schema, the attributes with low cardinality in each original dimension table are removed to form separate tables. These new tables are linked back to the original dimension table through artificial keys.

Advantages and Disadvantages

You may want to snowflake for one obvious reason. By eliminating all the long text fields from the dimension tables, you expect to save storage space. For example, if you have “men’s

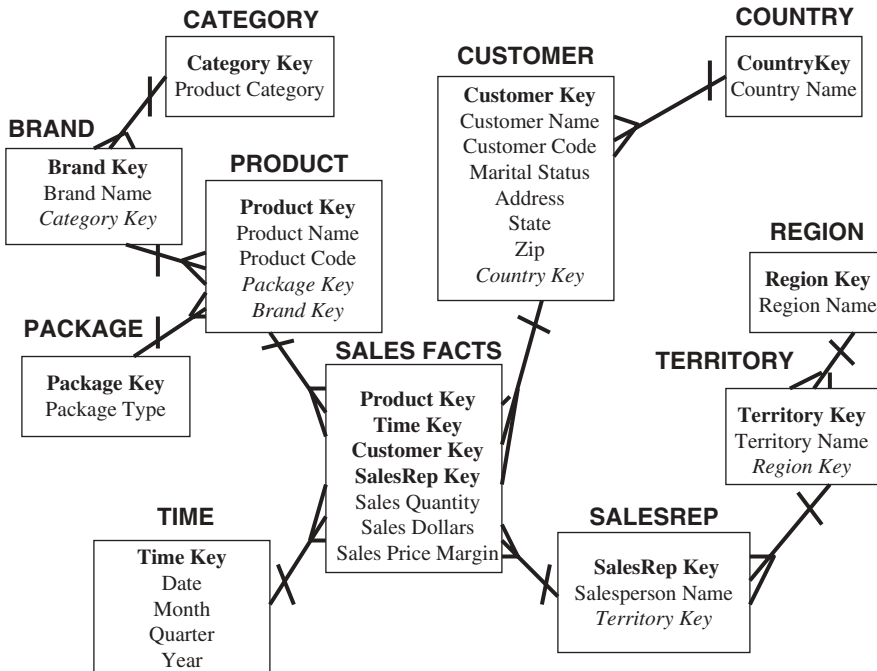


Figure 11-9 Sales: the “snowflake” schema.

furnishings” as one of the category names, that text will be repeated on every product row in that category. At first blush, removing such redundancies might appear to save significant storage space when the dimensions are large.

Let us assume that your product dimension table has 500,000 rows. By snowflaking you are able to remove 500,000 20-byte category names. At the same time, you have to add a 4-byte artificial category key to the dimension table. The net savings work out to be approximately 500,000 times 16, that is, about 8 MB. Your average 500,000-row product dimension table occupies about 200 MB of storage space and the corresponding fact table another 20 GB. The savings are just 4%. You will find that the small savings in space does not compensate for the other disadvantages of snowflaking.

Here is a brief summary of the advantages and limitations of snowflaking:

Advantages

- Small savings in storage space
- Normalized structures are easier to update and maintain

Disadvantages

- Schema less intuitive and end-users are put off by the complexity
- Ability to browse through the contents difficult
- Degraded query performance because of additional joins

Snowflaking is not generally recommended in a data warehouse environment. Query performance takes the highest significance in a data warehouse and snowflaking hampers the performance.

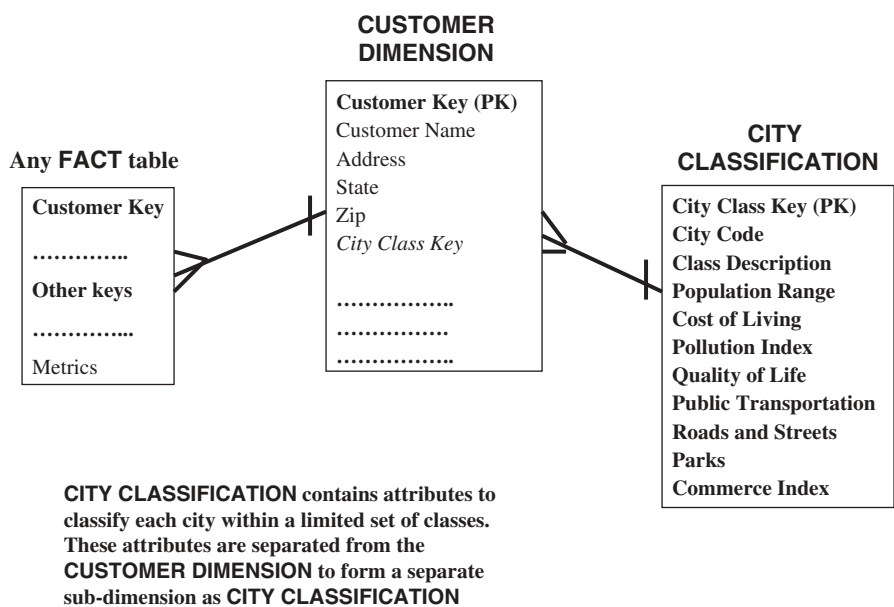


Figure 11-10 Forming a subdimension.

When to Snowflake

As an IT professional, you have an affinity for third normal form structures. We very well know all the problems unnormalized structures can cause. Further, wasted space could be another consideration for snowflaking.

In spite of the apparent disadvantages, are there any circumstances under which snowflaking may be permissible? The principle behind snowflaking is normalization of the dimension tables by removing low cardinality attributes and forming separate tables. In a similar manner, some situations provide opportunities to separate out a set of attributes and form a subdimension. This process is very close to the snowflaking technique. Figure 11-10 shows how a city classification subdimension is formed out of the customer dimension.

Although forming subdimensions may be construed as snowflaking, it makes a lot of sense to separate out the demographic attributes into another table. You usually load the demographic data at times different from the times when you load the other dimension attributes. The two sets of attributes differ in granularity. If the customer dimension is very large, running into millions of rows, the savings in storage space could be substantial. Another valid reason for separating out the demographic attributes relates to the browsing of attributes. Users may browse the demographic attributes more than the others in the customer dimension table.

AGGREGATE FACT TABLES

Aggregates are precalculated summaries derived from the most granular fact table. These summaries form a set of separate aggregate fact tables. You may create each aggregate fact table as a specific summarization across any number of dimensions. Let us begin by examining a sample STAR schema. Choose a simple STAR schema with the fact table at

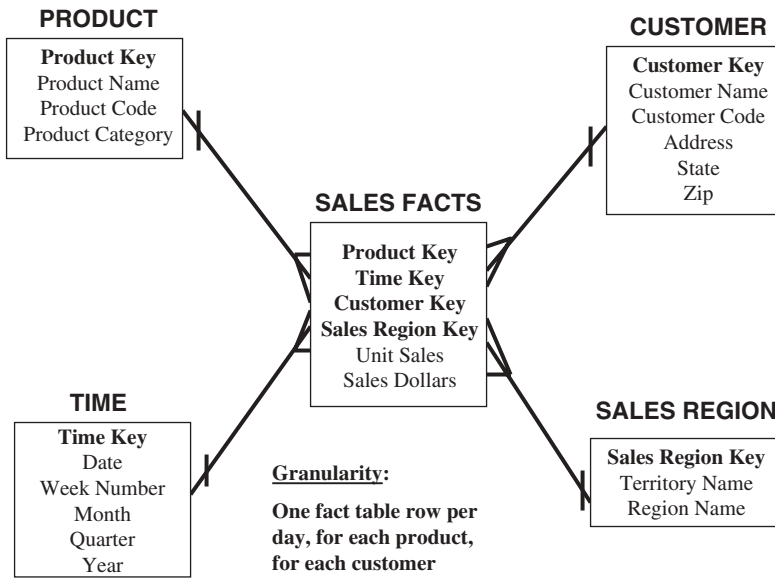


Figure 11-11 STAR schema with most granular fact table.

the lowest possible level of granularity. Assume there are four dimension tables surrounding this most granular fact table. Figure 11-11 shows the example we want to examine.

What is the essential difference between queries run in an operational system and those run in a data warehouse environment? When you run a query in an operational system, it produces a result set about a single customer, a single order, a single invoice, a single product, and so on. But, as you know, the queries in a data warehouse environment produce large result sets. Queries in a data warehouse retrieve hundreds and thousands of table rows, manipulate the metrics in the fact tables, and then produce the result sets. The manipulation of the fact table metrics may be a simple addition, an addition with some adjustments, a calculation of averages, or even an application of complex arithmetic algorithms.

Let us review a few typical queries against the sample STAR schema shown in Figure 11-11.

Query 1: Total sales for customer number 12345678 during the first week of December 2008 for product Widget-1.

Query 2: Total sales for customer number 12345678 during the first three months of 2009 for product Widget-1.

Query 3: Total sales for all customers in the south-central territory for the first two quarters of 2009 for product category Bigtools.

Scrutinize these queries and determine how the totals will be calculated in each case. The totals will be calculated by adding the sales quantities and sales dollars from the qualifying rows of the fact table. In each case, let us review the qualifying rows that contribute to the total in the result set.

Query 1: All fact table rows where the customer key relates to customer number 12345678, the product key relates to product Widget-1, and the time key relates to

the seven days in the first week of December 2008. Assuming that a customer may make at most one purchase of a single product in a single day, only a maximum of seven fact table rows participate in the summation.

Query 2: All fact table rows where the customer key relates to customer number 12345678, the product key relates to product Widget-1, and the time key relates to about 90 days of the first quarter of 2009. Assuming that a customer may make at most one purchase of a single product in a single day, only about 90 fact table rows or less participate in the summation.

Query 3: All fact table rows where the customer key relates to all customers in the south-central territory, the product key relates to all products in the product category Bigtools, and the time key relates to about 180 days in the first two quarters of 2009. In this case, clearly a large number of fact table rows participate in the summation.

Obviously, Query 3 will run long because of the large number of fact table rows to be retrieved. What can be done to reduce the query time? This is where aggregate tables can be helpful. Before we discuss aggregate fact tables in detail, let us review the sizes of some typical fact tables in real-world data warehouses.

Fact Table Sizes

Figure 11-12 represents the STAR schema for sales of a large supermarket chain. There are about two billion rows of the base fact table with the lowest level of granularity. Study the calculations shown below:

Time dimension: 5 years $\times$ 365 days = 1825

Store dimension: 300 stores reporting daily sales

Product dimension: 40,000 products in each store (about 4000 sell in each store daily)

Promotion dimension: a sold item may be in only one promotion in a store on a given day

Maximum number of base fact table records: $1825 \times 300 \times 4000 \times 1 = 2$ billion

Here are a few more estimates of the fact table sizes in other typical cases:

Telephone call monitoring

Time dimension: 5 years = 1825 days

Number of calls tracked each day: 150 million

Maximum number of base fact table records: 274 billion

Credit card transaction tracking

Time dimension: 5 years = 60 months

Number of credit card accounts: 150 million

Average number of monthly transactions per account: 20

Maximum number of base fact table records: 180 billion

From the above examples you see the typical enormity of the fact tables that are at the lowest level of granularity. Although none of the queries from the users would call for

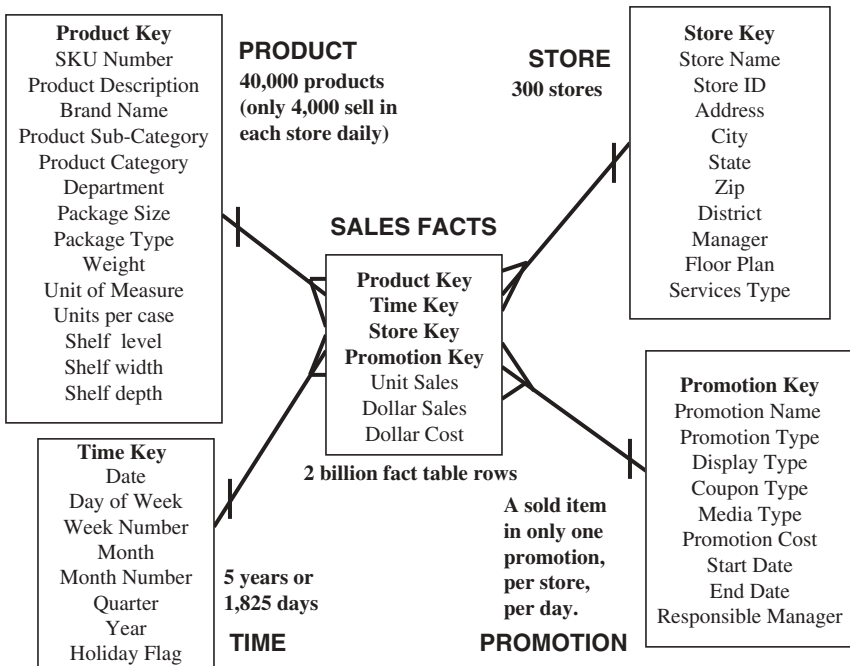


Figure 11-12 STAR schema: grocery chain.

data just from a single row in these fact tables, data at the lowest level of detail is needed. This is because when a user performs various forms of analysis, he or she must be able to get result sets comprising a variety of combinations of individual fact table rows. If you do not keep details by individual stores, you cannot retrieve result sets for products by individual stores. On the other hand, if you do not keep details by individual products, you cannot retrieve result sets for stores by individual products.

So, here is the question. If you need detailed data at the lowest level of granularity in the base fact tables, how do you deal with summations of huge numbers of fact table rows to produce query results? Consider the following queries related to a grocery chain data warehouse:

- How did the three new stores in Wisconsin perform during the last three months compared to the national average?
- What is the effect of the latest holiday sales campaign on meat and poultry?
- How do the July 4th holiday sales by product categories compare to last year?

Each of these three queries requires selections and summations from the fact table rows. For these types of summations, you need detailed data based on one or more dimensions, but only summary totals based on the other dimensions. For example, for the last query, you need detailed daily data based on the time dimension, but summary totals by product categories. In any case, if you had summary totals or precalculated aggregates readily available, the queries would run faster. With properly aggregated summaries, the performance of each of these queries can be dramatically improved.

Need for Aggregates

Refer to Figure 11-12 showing the STAR schema for a grocery chain. In those 300 stores, assume there are 500 products per brand. Of the 40,000 products, assume that there is at least one sale per product per store per week. Let us estimate the number of fact table rows to be retrieved and summarized for the following types of queries:

Query involves 1 product, 1 store, 1 week—retrieve/summarize only 1 fact table row

Query involves 1 product, all stores, 1 week—retrieve/summarize 300 fact table rows

Query involves 1 brand, 1 store, 1 week—retrieve/summarize 500 fact table rows

Query involves 1 brand, all stores, 1 year—retrieve/summarize 7,800,000 fact table rows

Suppose you had precalculated and created an aggregate fact table in which each row summarized the totals for a brand, per store, per week. Then the third query must retrieve only one row from this aggregate fact table. Similarly, the last query must retrieve only 15,600 rows from this aggregate fact table, much less than the 7 million rows.

Further, if you precalculate and create another aggregate fact table in which each row summarized the totals for a brand, per store, per year, the last query must retrieve only 300 rows.

Aggregates have fewer rows than the base tables. Therefore, when most of the queries are run against the aggregate fact tables instead of the base fact table, you notice a tremendous boost to performance in the data warehouse. Formation of aggregate fact tables is certainly a very effective method to improve query performance.

Aggregating Fact Tables

As we have seen, aggregate fact tables are merely summaries of the most granular data at higher levels along the dimension hierarchies. Figure 11-13 illustrates the hierarchies along three dimensions. Examine the hierarchies in the three dimensions. The hierarchy levels in the time dimension move up from day at the lowest level to year at the highest level. City is at the lowest level in the store dimension and product at the lowest level in the product dimension.

In the base fact table, the rows reflect the numbers at the lowest levels of the dimension hierarchies. For example, each row in the base fact table shows the sales units and sales dollars relating to one date, one store, and one product. By moving up one or more notches along the hierarchy in each dimension, you can create a variety of aggregate fact tables. Let us explore the possibilities.

Multi-Way Aggregate Fact Tables Figure 11-14 illustrates the different ways aggregate fact tables may be formed. The following paragraphs describe possible aggregates.

One-Way Aggregates When you rise to higher levels in the hierarchy of one dimension and keep the level at the lowest in the other dimensions, you create one-way aggregate tables. Review the following examples:

- Product category by store by date
- Product department by store by date

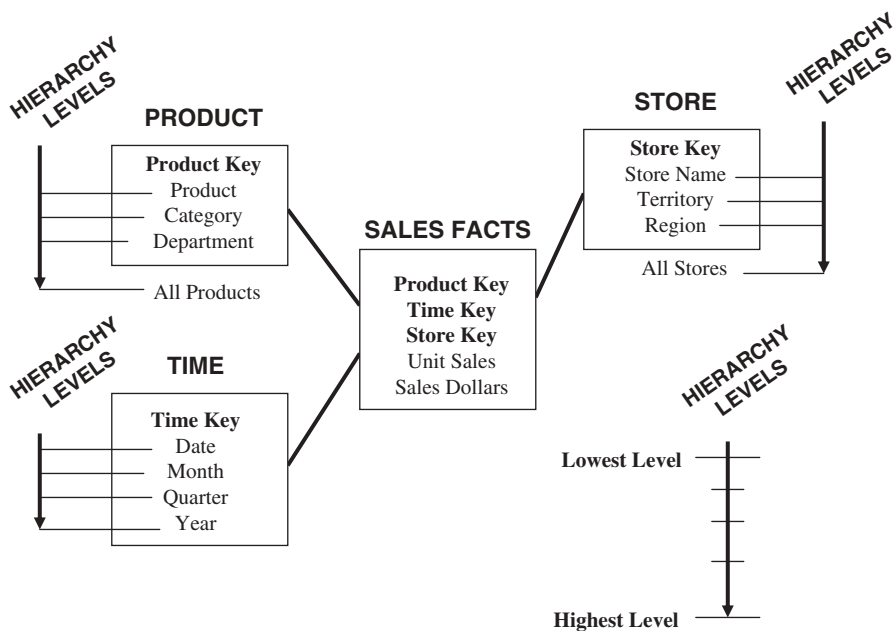


Figure 11-13 Dimension hierarchies.

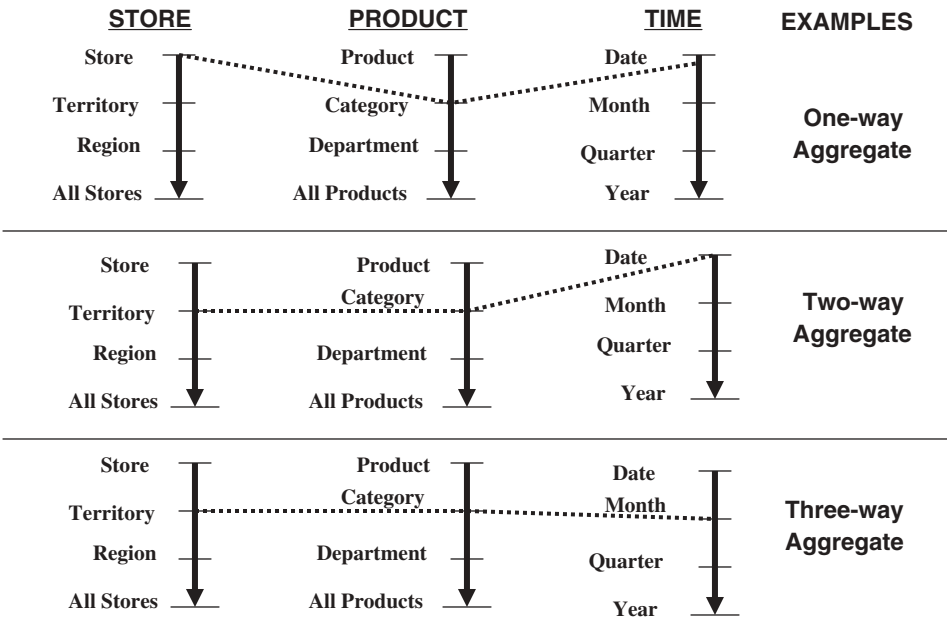


Figure 11-14 Forming aggregate fact tables.

- All products by store by date
- Territory by product by date
- Region by product by date
- All stores by product by date
- Month by store by product
- Quarter by store by product
- Year by store by product

Two-Way Aggregates When you rise to higher levels in the hierarchies of two dimensions and keep the level at the lowest in the other dimension, you create two-way aggregate tables. Review the following examples:

- Product category by territory by date
- Product category by region by date
- Product category by all stores by date
- Product category by month by store
- Product category by quarter by store
- Product category by year by store
- Product department by territory by date
- Product department by region by date
- Product department by all stores by date
- Product department by month by store
- Product department by quarter by store
- Product department by year by store
- All products by territory by date
- All products by region by date
- All products by all stores by date
- All products by month by store
- All products by quarter by store
- All products by year by store
- District by month by product
- District by quarter by product
- District by year by product
- Territory by month by product
- Territory by quarter by product
- Territory by year by product
- Region by month by product
- Region by quarter by product
- Region by year by product
- All stores by month by product
- All stores by quarter by product
- All stores by year by product

Three-Way Aggregates When you rise to higher levels in the hierarchies of all the three dimensions, you create three-way aggregate tables. Review the following examples:

- Product category by territory by month
- Product department by territory by month
- All products by territory by month
- Product category by region by month
- Product department by region by month
- All products by region by month
- Product category by all stores by month
- Product department by all stores by month
- Product category by territory by quarter
- Product department by territory by quarter
- All products by territory by quarter
- Product category by region by quarter
- Product department by region by quarter
- All products by region by quarter
- Product category by all stores by quarter
- Product department by all stores by quarter
- Product category by territory by year
- Product department by territory by year
- All products by territory by year
- Product category by region by year
- Product department by region by year
- All products by region by year
- Product category by all stores by year
- Product department by all stores by year
- All products by all stores by year

Each of these aggregate fact tables is derived from a single base fact table. The derived aggregate fact tables are joined to one or more derived dimension tables. Figure 11-15 shows a derived aggregate fact table connected to a derived dimension table.

Effect of Sparsity on Aggregation Consider the case of the grocery chain with 300 stores, 40,000 products in each store, but only 4000 selling in each store in a day. As discussed earlier, assuming that you keep records for 5 years or 1825 days, the maximum number of base fact table rows is calculated as follows:

Product = 40,000

Store = 300

Time = 1825

Maximum number of base fact table rows = 22 billion

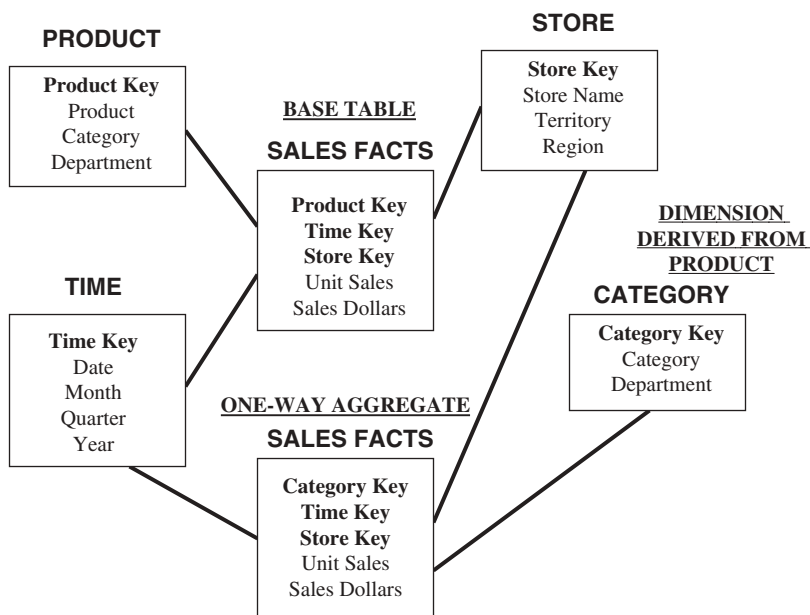


Figure 11-15 Aggregate fact table and derived dimension table.

Because only 4000 products sell in each store in a day, not all of these 22 billion rows are occupied. Because of this sparsity, only 10% of the rows are occupied. Therefore, the real estimate of the number of base table rows is 2 billion.

Now let us see what happens when you form aggregates. Scrutinize a one-way aggregate: brand totals by store by day. Calculate the maximum number of rows in this one-way aggregate.

Brand = 80

Store = 300

Time = 1825

Maximum number of aggregate table rows = 43,800,000

While creating the one-way aggregate, you will notice that the sparsity for this aggregate is not 10% as in the case of the base table. This is because when you aggregate by brand, more of the brand codes will participate in combinations with store and time codes. The sparsity of the one-way aggregate would be about 50%, resulting in a real estimate of 21,900,000. If the sparsity had remained as the 10% applicable to the base table, the real estimate of the number of rows in the aggregate table would be much less.

When you go for higher levels of aggregates, the sparsity percentage moves up and even reaches 100%. Because of the failure of sparsity to stay lower, you are faced with the question whether aggregates do improve performance that much. Do they reduce the number of rows that dramatically?

Experienced data warehousing practitioners have a suggestion. When you form aggregates, make sure that each aggregate table row summarizes at least 10 rows in the lower level table. If you increase this to 20 rows or more, it would be really remarkable.

Aggregation Options

Going back to our discussion of one-way, two-way, and three-way aggregates for a basic STAR schema with just three dimensions, you could count more than 50 different ways you may create aggregates. In the real world, the number of dimensions is not just three, but many more. Therefore, the number of possible aggregate tables escalates into the hundreds.

Further, from the reference to the failure of sparsity in aggregate tables, you know that the aggregation process does not reduce the number of rows proportionally. In other words, if the sparsity of the base fact table is 10%, the sparsity of the higher-level aggregate tables does not remain at 10%. The sparsity percentage increases more and more as your aggregate tables climb higher and higher in levels of summarization.

Is aggregation that much effective after all? What are some of the options? How do you decide what to aggregate? First, set a few goals for aggregation for your data warehouse environment.

Goals for Aggregation Strategy Apart from the general overall goal of improving data warehouse performance, here are a few specific, practical goals:

- Do not get bogged down with too many aggregates. Remember, you have to create additional derived dimensions as well to support the aggregates.
- Try to cater to a wide range of user groups. In any case, provide for your power users.
- Go for aggregates that do not unduly increase the overall usage of storage. Look carefully into larger aggregates with low sparsity percentages.
- Keep the aggregates hidden from the end-users but be transparent to the end-user query. That is, it is the query tool that needs to be aware of the aggregates to direct the queries for proper access.
- Attempt to keep the impact on the data staging process as minimally intensive as possible.

Practical Suggestions Before doing any calculations to determine the types of aggregates needed for your data warehouse environment, spend a good deal of time on determining the nature of the common queries. How do your users normally report results? What are the reporting levels? By stores? By months? By product categories? Go through the dimensions, one by one, and review the levels of the hierarchies. Check if there are multiple hierarchies within the same dimension. If so, find out which of these multiple hierarchies are more important. In each dimension, ascertain which attributes are used for grouping the fact table metrics. The next step is to determine which of these attributes are used in combinations and what the most common combinations are.

Once you determine the attributes and their possible combinations, look at the number of values for each attribute. For example, in a hotel chain schema, assume that hotel is at the lowest level and city is at the next higher level in the hotel dimension. Let us say there are 25,000 values for hotel and 15,000 values for city. Clearly, there is no big advantage of aggregating by cities. On the other hand, if city has only 500 values, then city is a level at which you may consider aggregation. Examine each attribute in the hierarchies within a dimension. Check the values for each of the attributes. Compare the values of attributes at different levels of the same hierarchy and decide which ones are strong candidates to participate in aggregation.

Develop a list of attributes that are useful candidates for aggregation, then work out the combinations of these attributes to come up with your first set of multi-way aggregate fact tables. Determine the derived dimension tables you need to support these aggregate fact tables. Go ahead and implement these aggregate fact tables as the initial set.

Bear in mind that aggregation is a performance tuning mechanism. Improved query performance drives the need to summarize, so do not be too concerned if your first set of aggregate tables do not perform perfectly. Your aggregates are meant to be monitored and revised as necessary. The nature of the bulk of the query requests is likely to change. As your users become more adept at using the data warehouse, they will devise new ways of grouping and analyzing data. So what is the practical advice? Do your preparatory work. Start with a reasonable set of aggregate tables, and continue to make adjustments as necessary.

FAMILIES OF STARS

When you look at a single STAR schema with its fact table and the surrounding dimension tables, you know that is not the extent of a data warehouse. Almost all data warehouses contain multiple STAR schema structures. Each STAR serves a specific purpose to track the measures stored in the fact table. When you have a collection of related STAR schemas, you may call the collection a family of STARS. Families of STARS are formed for various reasons. You may form a family by just adding aggregate fact tables and the derived dimension tables to support the aggregates. Sometimes, you may create a core fact table containing facts interesting to most users and customized fact tables for specific user groups. Many factors lead to the existence of families of STARS. First, look at the example provided in Figure 11-16.

The fact tables of the STARS in a family share dimension tables. Usually, the time dimension is shared by most of the fact tables in the group. In the above example, all three fact tables are likely to share the time dimension. Going the other way, dimension tables from multiple STARS may share the fact table of one STAR.

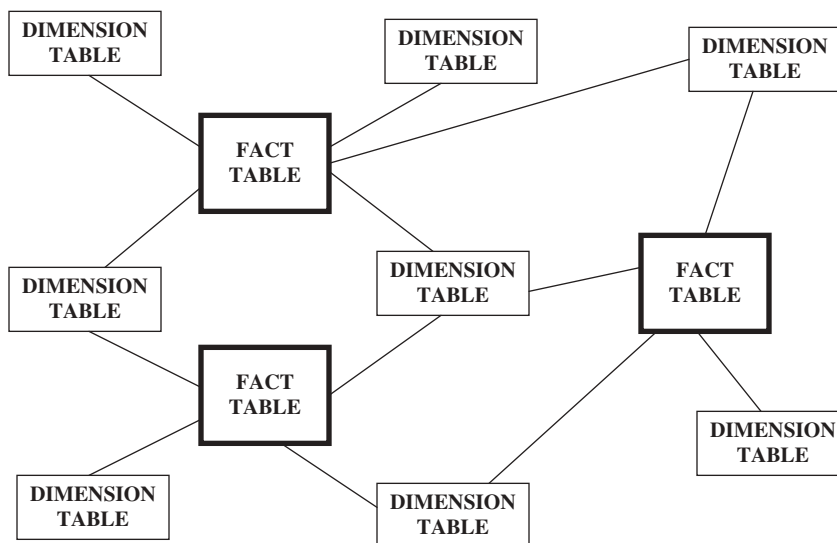


Figure 11-16 Family of STARS.

If you are in a business like banking or telephone services, it makes sense to capture individual transactions as well as snapshots at specific intervals. You may then use families of STARS consisting of transaction and snapshot schemas. If you are in a manufacturing company or a similar production-type enterprise, your company needs to monitor the metrics along the value chain. Some other institutions are like a medical center, where value is added not in a chain but at different stations within the enterprise. For these enterprises, the family of STARS supports the value chain or the value circle. We will get into details in the next few sections.

Snapshot and Transaction Tables

Let us review some basic requirements of a telephone company. A number of individual transactions make up a telephone customer’s account. Many of the transactions occur during the hours of 6:00 a.m. to 10:00 p.m. of the customer’s day. More transactions happen during the holidays and weekends for residential customers. Institutional customers use the phones on weekdays rather than over the weekends. A telephone company accumulates a very large collection of rich transaction data that can be used for many types of valuable analysis. The telephone company needs a schema capturing transaction data that supports strategic decision making for expansions, new service improvements, and so on. This transaction schema answers questions such as how does the revenue of peak hours over the weekends and holidays compare with peak hours over weekdays.

In addition, the telephone company needs to answer questions from the customers as to account balances. The customer service departments are constantly bombarded with questions on the status of individual customer accounts. At periodic intervals, the accounting department may be interested in the amounts expected to be received by the middle of next month. What are the outstanding balances for which bills will be sent this month-end? For these purposes, the telephone company needs a schema to capture snapshots at periodic intervals. Figure 11-17 shows the snapshot and transaction fact tables for a

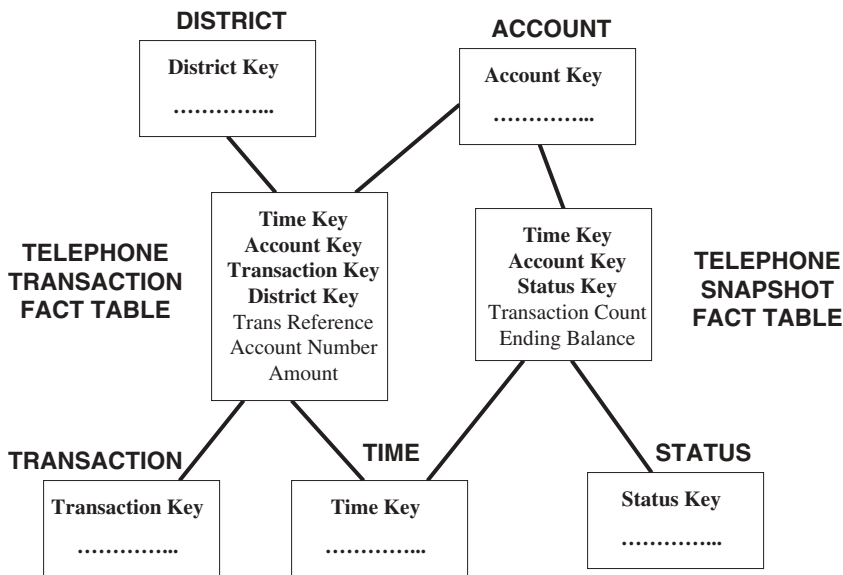


Figure 11-17 Snapshot and transaction tables.

telephone company. Make a note of the attributes in the two fact tables. One table tracks the individual phone transactions. The other table holds snapshots of individual accounts at specific intervals. Also, notice how dimension tables are shared between two fact tables.

Snapshots and transaction tables are also common for banks. For example, an ATM transaction table stores individual ATM transactions. This fact table keeps track of individual transaction amounts for the customer accounts. The snapshot table holds the balance for each account at the end of each day. The two tables serve two distinct functions. From the transaction table, you can perform various types of analysis of the ATM transactions. The snapshot table provides total amounts held at periodic intervals showing the shifting and movement of balances.

Financial data warehouses also require snapshot and transaction tables because of the nature of the analysis in these cases. The first set of questions for these warehouses relates to the transactions affecting given accounts over a certain period of time. The other set of questions centers around balances in individual accounts at specific intervals or totals of groups of accounts at the end of specific periods. The transaction table answers the questions of the first set; the snapshot table handles the questions of the second set.

Core and Custom Tables

Consider two types of businesses that are apparently dissimilar. First take the case of a bank. A bank offers a large variety of services all related to finance in one form or another. Most of the services are different from one another. The checking account service and the savings account service are similar in most ways. But the savings account service does not resemble the credit card service in any way. How do you track these dissimilar services?

Next, consider a manufacturing company producing a number of heterogeneous products. Although a few factors may be common to the various products, by and large the factors differ. What must you do to get information about heterogeneous products?

A different type of the family of STARS satisfies the requirements of these companies. In this type of family, all products and services connect to a core fact table and each product or service relates to individual custom tables. In Figure 11-18, you will see the core and custom tables for a bank. Note how the core fact table holds the metrics that are common to all types of accounts. Each custom fact table contains the metrics specific to that line of service. Also note the shared dimension and notice how the tables form a family of STARS.

Supporting Enterprise Value Chain or Value Circle

In a manufacturing business, a product travels through various steps, starting off as raw materials and ending as finished goods in the warehouse inventory. Usually, the steps include addition of ingredients, assembly of materials, process control, packaging, and shipping to the warehouse. From finished goods inventory, a product moves into shipments to distributor, distributor inventory, distributor shipment, retail inventory, and retail sales. At each step, value is added to the product. Several operational systems support the flow through these steps. The whole flow forms the supply chain or the value chain. Similarly, in an insurance company, the value chain may include a number of steps from sales of insurance through issuance of policy and then finally claims processing. In this case, the value chain relates to the service.

If you are in one of these businesses, you need to track important metrics at different steps along the value chain. You create STAR schemas for the significant steps and the complete

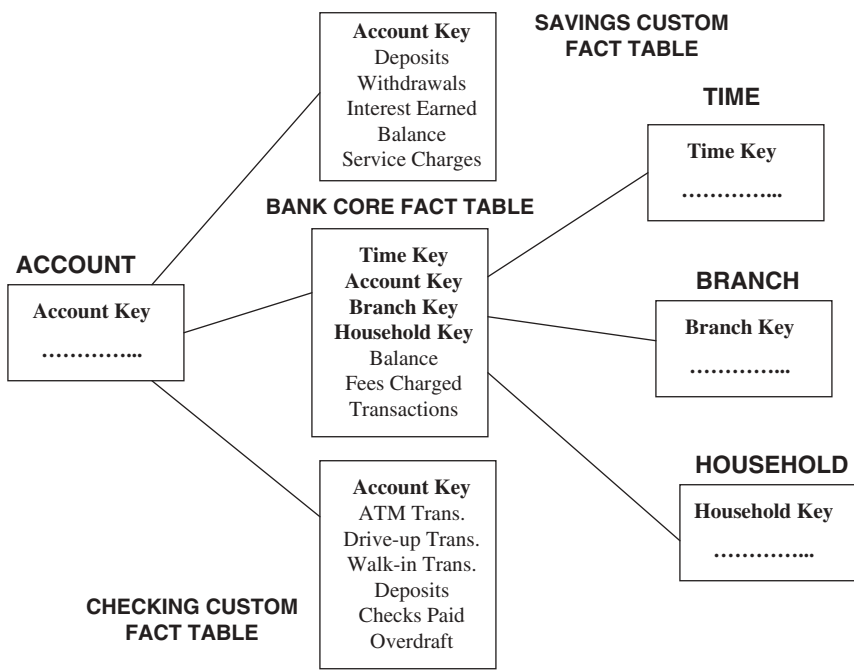


Figure 11-18 Core and custom tables.

set of related schemas forms a family of STARS. You define a fact table and a set of corresponding dimensions for each important step in the chain. If your company has multiple value chains, then you have to support each chain with a separate family of STARS.

A supply chain or a value chain runs in a linear fashion beginning with a certain step and ending at another step with many steps in between. Again, at each step, value is added. In some other kinds of businesses where value gets added to services, similar linear movements do not exist. For example, consider a health care institution where value gets added to patient service from different units almost as if they form a circle around the service. We perceive a value circle in such organizations. The value circle of a large health maintenance organization may include hospitals, clinics, doctor's offices, pharmacies, laboratories, government agencies, and insurance companies. Each of these units either provides patient treatments or measures patient treatments. Patient treatment by each unit may be measured in different metrics. But most of the units would analyze the metrics using the same set of conformed dimensions such as time, patient, health care provider, treatment, diagnosis, and payer. For a value circle, the family of STARS comprises multiple fact tables and a set of conformed dimensions.

Conforming Dimensions

While exploring families of STARS, you will have noticed that dimensions are shared among fact tables. Dimensions form common links between STARS. For dimensions to be conformed, you have to deliberately make sure that common dimensions may be used between two of more STARS. If the product dimension is shared between two fact tables of sales and inventory, then the attributes of the product dimension must have the same meaning in relation to each of the two fact tables. Figure 11-19 shows a set of conformed dimensions.

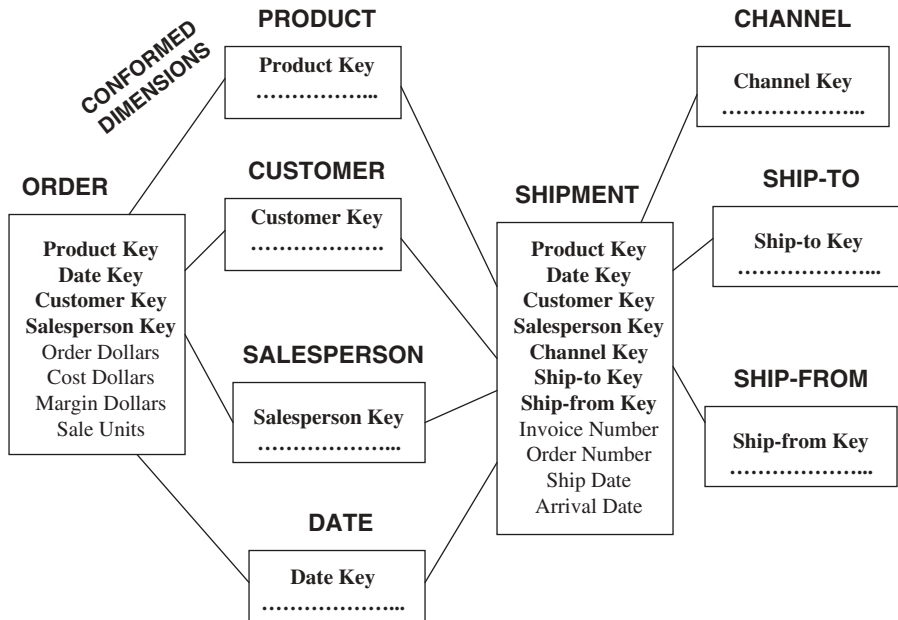


Figure 11-19 Conformed dimensions.

The order and shipment fact tables share the conformed dimensions of product, date, customer, and salesperson. A conformed dimension is a comprehensive combination of attributes from the source systems after resolving all discrepancies and conflicts. For example, a conformed product dimension must truly reflect the master product list of the enterprise and must include all possible hierarchies. Each attribute must be of the correct data type and must have proper lengths and constraints.

Conforming dimensions is a basic requirement in a data warehouse. Pay special attention and take the necessary steps to conform all your dimensions. This is a major responsibility of the project team. Conformed dimensions allow roll ups across data marts. User interfaces will be consistent irrespective of the type of query. Result sets of queries will be consistent across data marts. Of course, a single conformed dimension can be used against multiple fact tables.

Standardizing Facts

In addition to the task of conforming dimensions is the requirement to standardize facts. We have seen that fact tables work across STAR schemas. Review the following issues relating to the standardization of fact table attributes:

- Ensure the same definitions and terminology across data marts.
- Resolve homonyms and synonyms.
- Types of facts to be standardized include revenue, price, cost, and profit margin.
- Guarantee that the same algorithms are used for any derived units in each fact table.
- Make sure each fact uses the right unit of measurement.

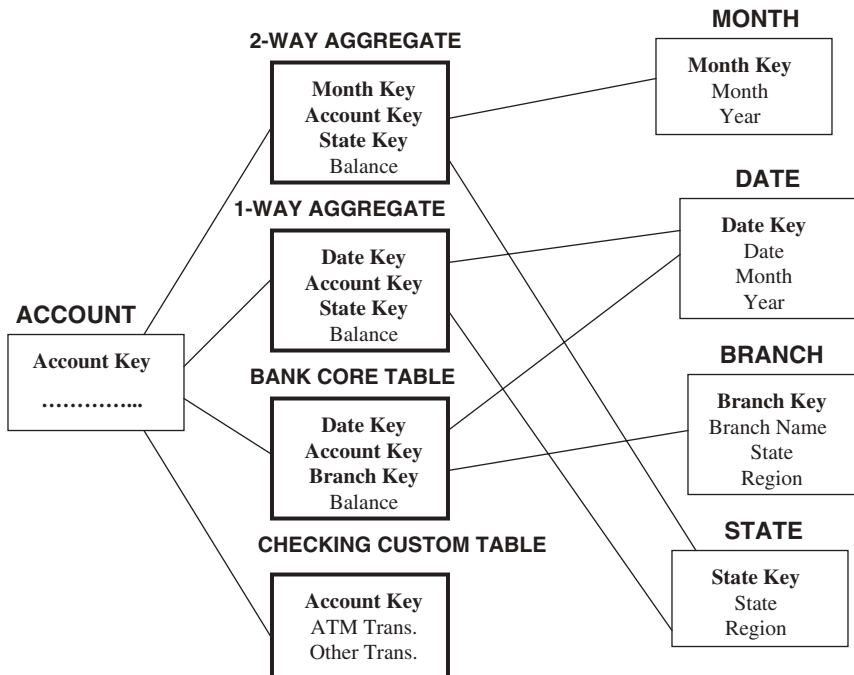


Figure 11-20 A comprehensive family of STARS.

Summary of Family of STARS

Let us end our discussion of the family of STARS with a comprehensive diagram showing a set of standardized fact tables and conformed dimension tables. Study Figure 11-20 carefully. Note the aggregate fact tables and the corresponding derived dimension tables. What types of aggregates are these? Which are the base fact tables? Notice the shared dimensions. Are these conformed dimensions? See how the various fact tables and the dimension tables are related.

CHAPTER SUMMARY

- Slowly changing dimensions may be classified into three different types based on the nature of the changes. Type 1 relates to corrections, type 2 to preservation of history, and type 3 to soft revisions. Applying each type of revision to the data warehouse is different.
- Large dimension tables such as customer or product need special considerations for applying optimizing techniques.
- “Snowflaking” or creating a snowflake schema is a method of normalizing the STAR schema. Although some conditions justify the snowflake schema, it is generally not recommended.
- Miscellaneous flags and textual data are thrown together in one table called a junk dimension table.

- Aggregate or summary tables improve performance. Formulate a strategy for building aggregate tables.
- A set of related STAR schemas make up a family of STARS. Examples are snapshot and transaction tables, core and custom tables, and tables supporting a value chain or a value circle. A family of STARS relies on conformed dimension tables and standardized fact tables.

REVIEW QUESTIONS

1. Describe slowly changing dimensions. What are the three types? Explain each type very briefly.
2. Compare and contrast type 2 and type 3 slowly changing dimensions.
3. Can you treat rapidly changing dimensions in the same way as type 2 slowly changing dimensions? Discuss.
4. What are junk dimensions? Are they necessary in a data warehouse?
5. How does a snowflake schema differ from a STAR schema? Name two advantages and two disadvantages of the snowflake schema.
6. Differentiate between slowly and rapidly changing dimensions.
7. What are aggregate fact tables? Why are they needed? Give an example.
8. Describe with examples snapshot and transaction fact tables. How are they related?
9. Give an example of a value circle. Explain how a family of STARS can support a value circle.
10. What is meant by conforming the dimension? Why is this important in a data warehouse?

EXERCISES

1. Indicate if true or false:
 - A. Type 1 changes for slowly changing dimensions relate to correction of errors.
 - B. To apply type 3 changes of slowly changing dimensions, overwrite the attribute value in the dimension table row with the new value.
 - C. Large dimensions usually have multiple hierarchies.
 - D. The STAR schema is a normalized version of the snowflake schema.
 - E. Aggregates are precalculated summaries.
 - F. The percentage of sparsity of the base table tends to be higher than that of aggregate tables.
 - G. The fact tables of the STARS in a family share dimension tables.
 - H. Core and custom fact tables are useful for companies with several lines of service.
 - I. Conforming dimensions is not absolutely necessary in a data warehouse.
 - J. A value circle usually needs a family of STARS to support the business.

2. Assume you are in the insurance business. Find two examples of type 2 slowly changing dimensions in that business. As an analyst on the project, write the specifications for applying the type 2 changes to the data warehouse with regard to the two examples.
3. You are the data design specialist on the data warehouse project team for a retail company. Design a STAR schema to track the sales units and sales dollars with three dimension tables. Explain how you will decide to select and build four two-way aggregates.
4. As the data designer for an international bank, consider the possible types of snapshot and transaction tables. Complete the design of one snapshot and one transaction table.
5. For a manufacturing company, design a family of three STARS to support the value chain.